Build with AI

A Practical Guide for Developers, Product Managers, and Business Leaders

12 Parts · Complete Series · 2025

Table of Contents

Part 0

Build with AI: A Practical Guide for Non-Developers

9 min read · Introduction

Introducing the "Build with AI" series

> **In short:** *Build with AI* is a free, 12-part series for non-developers — founders, operations managers, consultants, and domain experts — who want to build real, working things with AI, not just chat with it. No coding background is required. The one prerequisite is a willingness to think carefully about your problem, your data, and your users.

Something has shifted.

Not in the technology itself — AI has been improving for years. The shift is in who it's for.

For most of its recent history, working meaningfully with AI required technical fluency. You needed to understand APIs, training pipelines, model architectures. The people who got real value from it were engineers, researchers, data scientists. Everyone else got a chat interface and was told to "explore."

That era is over.

In 2026, the tools have matured to the point where a consultant, a founder, a teacher, a small business owner — anyone with a clear problem and the willingness to think carefully about it — can build software, automate workflows, and deploy AI-powered solutions that would have required an engineering team two years ago.

The gap is no longer technical ability. The gap is understanding. Most people don't know how to think about AI — what it actually does, where it genuinely helps, where it reliably fails, how to get consistent results from it. They've seen the demos. They've tried the tools. They're stuck somewhere between impressed and frustrated.

This series is the bridge.

What this series is

"Build with AI" is a twelve-post series written for people who want to build real things with AI — not just use it as a fancy search engine or a slightly better autocomplete.

It's written for the founder who has a vision but not a technical co-founder. The operations manager who sees the same inefficiency every day and knows software could fix it. The consultant who wants to build the exact tool their clients need. The teacher, the designer, the domain expert of any kind who understands a problem deeply and wants to translate that understanding into something that works.

You don't need to know how to code to read this series. You don't need to have used AI beyond basic chat. You need to be willing to think carefully — about your problems, your data, your users, what you actually want to build.

That's the prerequisite.

What this series is not

It's not a prompt hack collection. It's not a list of "top 10 AI tools." It's not hype about what AI will be able to do someday.

Every post is grounded in what's actually working right now, in 2026, with tools real people are using to build real things. Where the technology has genuine limits, we name them honestly. Where something is more hype than substance, we say so.

The goal is not to make you excited about AI. The goal is to make you effective with it.

Who wrote this — and why

This series draws from *AI Development Guide* by Jaehee Song — a practitioner's book written by someone who has spent over twenty years building enterprise data platforms, has taught AI development and vibe coding to hundreds of students, and has helped Korean technology startups navigate the US market through Seattle Partners.

The book was written because of a specific frustration: the existing resources were either too technical (written for engineers) or too shallow (written for people who just want tips). There was no complete, honest, practical framework for the builder who is not a developer but wants to build seriously.

This series distills the core of that framework — post by post, topic by topic — with enough practical depth that each one is worth reading on its own, and enough breadth that reading all twelve gives you a complete map of the territory.

The twelve posts

Here's where we're going:

Understanding AI (Posts 1–3)

Post 1 — Why AI feels frustrating The real reason most people aren't getting results: a mismatch between how they think AI works and how it actually works. The mental model shift that changes everything — plus the comparison spiral that makes people feel left behind and what to do about it.

Post 2 — What AI actually does (and doesn't do) An honest, clear-eyed breakdown: where AI genuinely excels (language transformation, pattern synthesis, rule-based data processing, finding things in large bodies of information) and where it reliably fails (factual accuracy without grounding, novel reasoning, consequential judgment). Updated with MCP tool connections and chain-of-thought reasoning capabilities.

Post 3 — The AI Lego Stack How modern AI tools fit together in three layers — the Model (the brain), the Orchestration (the coordinator), and the Interface (the front door). Which models are current in 2026, how to pick your stack, and why the model is the least important choice you'll make.

Working with Data and Prompts (Posts 4–6)

Post 4 — Your data is the real bottleneck Why the quality of what you feed AI determines the quality of what comes out — more than which model you use. The four data problems that kill AI projects (inconsistency, incompleteness, staleness, context collapse) and a practical data readiness checklist.

Post 5 — The real skill isn't coding In an era where Cursor, Bolt, Lovable, and Claude Code can build almost anything you can describe — the bottleneck has shifted entirely to how precisely you can define what you want. The problem definition framework and why ten minutes of clarity here saves hours of rebuilding later.

Post 6 — Prompting is programming The five core prompt patterns that cover 80% of what makes a prompt effective: Role + Task + Context, Format Specification, Constraints and Anti-Patterns, Chain of Thought, and Few-Shot Examples. With before/after examples for each.

Building (Posts 7–9)

Post 7 — AI agents What makes something an agent (not just a chatbot), the three levels of agents, how to build your first one with n8n, and where the field is right now — agent swarms, AI employees with payment and phone capabilities, computer-using agents, and how MCP connects it all.

Post 8 — Vibe coding How to build a real app using plain English — the tools (Bolt, Lovable, Cursor, Claude Code, Windsurf, Replit), the five-phase workflow that actually produces results, a step-by-step walkthrough building a lead tracker, and the honest limits of what vibe coding currently handles well.

Post 9 — AI code assistants The bridge between vibe coding and professional development: Cursor, Windsurf, GitHub Copilot, Claude Code, OpenAI Codex, Google Antigravity. Five practical use cases for non-developers, the workflow that actually works, and power user tips including Plan Mode, CLAUDE.md files, model tiering, and when to reset vs. push through.

Shipping and Responsibility (Posts 10–11)

Post 10 — From demo to production Why "it worked in the demo" is not the same as "it works in production." The five stages of production-readiness: pilot, reliability engineering, cost architecture, scaling, and the production mindset. With real cost numbers, a monitoring toolkit, and a pre-launch checklist.

Post 11 — Risk, hallucination and responsible AI What builders need to know about AI's failure modes: the four hallucination types, the risk spectrum from low to high stakes, and the practical safeguard toolkit (grounding with RAG, confidence signaling, scope limiting, output validation, audit logging). Plus the legal and ethical landscape in 2026.

Looking Forward (Post 12)

Post 12 — The next wave Where the technology is actually heading: autonomous agent networks, physical AI and humanoid robots (what's working now vs. what's still years away), self-improving AI on the horizon, and what this means for builders today. Grounded in current 2026 data, not speculation.

How to read this series

If you're completely new to AI: Start at Post 1 and read in order. Each post builds on the previous ones.

If you've been using AI but aren't getting consistent results: Posts 1, 5, and 6 are the highest-leverage for you. The mental model, the problem definition, and the prompting patterns.

If you want to automate something specific: Posts 4, 7, and 8 are your path. Data readiness → agents → building.

If you're about to ship something: Posts 10 and 11 are essential. Production readiness and responsible building.

If you want to understand where things are heading: Post 12 first, then work backwards.

Each post is designed to be a complete, standalone read — you can drop into any one and get value without having read the others. But together they form a complete framework, and the connections between them are part of what makes them useful.

A note on the pace of change

AI is moving fast. Model versions change. Tools emerge and mature. Capabilities that seem miraculous today will be table stakes in six months.

The fundamentals don't change.

How to think about problems. How to define them precisely. How to evaluate data quality. How to prompt effectively. How to build in layers and test as you go. How to design for reliability and build responsibly. How to understand the shape of what you're building without needing to understand every line of code.

These are the things this series is actually about. The specific tools named throughout are accurate as of mid-2026 — but the underlying thinking is durable. Even if every model name in this series is outdated by the time you read it, the framework will still work.

Let's begin

The demo gap — the distance between what you see AI doing in viral videos and what you're getting from it yourself — is real. But it's closeable. The people on the other side of that gap aren't using different tools. They've just learned to think about

problems differently, communicate with AI more precisely, and build with more discipline.

That's learnable. That's what this series teaches.

Post 1 starts with the frustration — because that's where most people actually are — and explains exactly why it's happening and what to do about it.

Let's go.

"Build with AI" is available in English and Korean. The series draws from the AI Development Guide by Jaehee Song, available on Apple Books, Google Play Books, and all major platforms.

🛒 Amazon 📱 Apple Books ▶️ Google Play Books 🌐 All Platforms (Books2Read)

Part 1

Why AI Feels Frustrating — And the Mental Shift That Changes Everything

10 min read · Understanding AI

Part 1 of the "Build with AI" series

> **In short:** AI feels frustrating because most people treat it like a vending machine — request in, finished answer out. It actually behaves like a brilliant but context-blind collaborator who needs a good brief. The viral demos you envy hide their failed iterations and the precise briefing behind them. The fix isn't a better tool; it's clearer thinking.

You've seen the posts. Someone on LinkedIn built a full SaaS product over a weekend. A Twitter thread shows jaw-dropping results from a single prompt. A YouTube demo has AI writing code, designing interfaces, and deploying an app in real time — in under ten minutes.

Then you open ChatGPT or Claude, type in what you need, and get back... something. Slightly off. A bit generic. Confidently wrong on the details. You tweak the prompt. Try again. Get something marginally better. Eventually you close the tab — not with a solution, but with a vague sense of disappointment and the quiet feeling that everyone else has figured out something you haven't.

And now there are two frustrations happening at once.

The first is the output itself — it didn't do what you needed. The second is harder to admit: *everyone else seems to be getting incredible results, and I'm not. Am I doing something wrong? Am I already behind?*

That second frustration is the more dangerous one. Because it makes you feel like the problem is you — your skills, your intelligence, your ability to "get" AI. And that feeling makes people either over-invest (frantically learning every new tool) or give up entirely ("AI is just hype").

Here's what's actually going on.

The demo gap is real — and deliberately invisible

The viral AI demos you see are real. The results they show are genuinely possible. But what you don't see is the invisible work that made them possible: the precise brief, the five iterations before the one they filmed, the person behind the screen who deeply understood what they wanted before they ever typed a word.

What gets shared is the output. What doesn't get shared is the thinking that preceded it.

This creates a distorted picture. You see the magician's final trick, not the years of practice. You see the "built in a weekend" product, not the founder's years of domain expertise that let them define exactly what to build. You see the perfect prompt, not the ten that failed before it.

The gap between what you see in demos and what you're getting isn't a gap in your AI ability. It's a gap in context — and context is something you can build.

The wrong mental model: AI as a vending machine

Most people approach AI like a vending machine. You put in a request, you expect a specific output to drop out. When it doesn't, you're frustrated — and surrounded by people who seem to have cracked the code, the frustration compounds.

But AI doesn't work like a vending machine. It works more like a **brilliant but context-blind collaborator** who just joined your team today.

Think about what you'd do if you hired an incredibly talented person — one who has read every book, learned every framework, speaks every language — but has never met you, doesn't know your project, your standards, or your audience, and has no memory of your last conversation.

Would you walk up to them and say: *"Write the report"*?

Of course not. You'd brief them. You'd explain the context, the goal, the audience, what "good" looks like. You'd give examples. You'd iterate together.

That's the shift. **AI isn't a button. It's a collaborator who needs a good brief.**

And when you see someone getting extraordinary AI results? They've gotten good at writing that brief. That's the skill — and it's learnable.

Why this changes everything in practice

Once you make this shift, everything about how you work with AI changes.

Before the shift, you type: *"Write me a marketing email for my product."* You get something bland and generic. You're disappointed. You go back to LinkedIn and see another post about someone's AI breakthrough. The spiral tightens.

After the shift, you think: *What does my collaborator need to know to do this well?*

So instead you write:

> "I'm writing a marketing email for a Seattle-based consulting firm that helps Korean tech startups enter the US market. Our audience is startup founders aged 30–45 who are skeptical of consultants but open to peer recommendations. The tone should be direct, slightly informal, and confident — not salesy. The goal is to get them to book a 30-minute call. Here's a similar email we've sent before that performed well: [example]."

Same tool. Completely different result.

The quality of your AI output is almost always a direct reflection of the quality of your input. Not your technical skill. Not which tool you use. Not whether you've taken the right course or watched the right YouTube video. **The quality of your thinking about the problem.**

The comparison trap has a specific shape

It's worth naming this clearly, because the comparison pressure around AI has a particular pattern that makes it especially corrosive.

AI is moving fast — genuinely fast. New tools, new capabilities, new benchmarks every few weeks. This creates a specific anxiety: not just "I'm not good at this" but "I'm falling behind something that's accelerating." It feels like trying to board a train that's already moving, while watching other people film themselves jumping on effortlessly.

But here's what the acceleration actually means for you: **the tools are getting easier faster than you're falling behind.**

The Cursor of today is dramatically easier to use than the Cursor of a year ago. Claude, ChatGPT, Gemini — all of them have gotten better at understanding

incomplete, imperfect prompts. The floor keeps rising. The person who starts today with the right mental model will outperform someone who started a year ago with the wrong one.

What doesn't change — what actually compounds over time — is your clarity of thinking. Your ability to define problems precisely. Your domain knowledge. Your understanding of what you want to build and why.

That's the asset worth developing. Not keeping up with every new tool release.

A simple exercise: the briefing test

Before you send your next AI prompt, ask yourself three questions:

1. Would a smart new colleague understand what I'm actually asking for? Not a keyword search — a human colleague. If your prompt would make sense as a Google search but not as a task you'd hand to a person, it needs more context.

2. Have I told them what "good" looks like? Constraints are a gift. "Make it short" is not a constraint. "Write this for a founder who has 2 minutes on a commute — 150 words max, first sentence must hook them" is a constraint. AI performs dramatically better when it knows what success looks like.

3. Have I given an example or a reference? The fastest way to close the gap between what you imagine and what AI produces is to show it something close to what you want. A sample output. A tone reference. A structure to follow. Examples are worth more than paragraphs of description.

Run your last 3 AI prompts through this test. You'll immediately see why some worked and most didn't.

AI as a thinking partner, not a shortcut

There's a subtler trap: using AI to skip thinking. You have a problem, you hand it off as-is, hoping it comes back solved.

This almost never works. Because the hard part of most work isn't execution — it's clarity. What exactly is the problem? What does a good solution look like? What constraints matter?

Try this: before asking AI to *do* something, ask it to help you *define* the problem first.

> "I'm trying to improve my team's weekly reporting process. Before we build anything, help me think through what's actually frustrating about the current process and what a better version might look like."

You'll often find the problem you thought you had isn't the real problem. And now you have a much sharper brief for whatever comes next.

The tools aren't the bottleneck

In 2026, Cursor, Claude, Bolt, Lovable — all evolving at a pace that would have seemed impossible three years ago. The ability to execute is cheaper and faster than it has ever been.

The bottleneck is clarity. The ability to define what you want to build, for whom, and why — precisely enough that a powerful tool can act on it.

This is good news if you're not a developer. Clarity is a thinking skill, not a technical skill. It's a skill you already have the foundation for — and one you can sharpen deliberately.

Stop watching the demos. Stop measuring yourself against LinkedIn posts. The people getting extraordinary results from AI aren't using a different tool than you. They've just learned to think out loud more precisely.

That's the whole game. And that's where this series starts.

Your first action

Take one thing you've been wanting to use AI for — something you tried and felt frustrated by, or something you've been putting off.

Don't open the AI tool yet.

First, write down:

- What exactly do you want to create or accomplish?
- Who is it for, and what do they need?
- What does a good result look like? What would make you say "yes, that's it"?
- Is there any example, reference, or sample you could point to?

Now open the tool. See the difference.

What to remember from this post

The viral results you see online are real, but the invisible brief, the failed iterations, and the domain clarity that made them possible are never shown. You're not behind. You're just seeing the highlight reel.

Stop expecting an output to drop out of a request. Start thinking: *what does my collaborator need to know to do this well?* That single shift changes everything.

The quality of your output mirrors the quality of your thinking, not your technical skill or which tool you use. And that clarity compounds — every time you use it, you get sharper at defining problems, understanding your audience, and communicating what "good" looks like. The feeling of being left behind is real, but it's not caused by a gap in ability. It's caused by a gap in approach. And approach is something you can change today.

Want the full framework?

This post covers the foundational mindset shift. The *AI Development Guide* by Jaehee Song goes much deeper — from how AI actually thinks (strengths, failure modes, why hallucinations happen) through to building production-ready solutions without writing code.

If you've felt the gap between what AI can supposedly do and what you're actually getting from it, this book is the bridge.

🛒 Amazon 📱 Apple Books ▶️ Google Play Books 🌐 All Platforms (Books2Read)

Next in the series: "What AI Actually Does (and Doesn't Do)" — a clear-eyed look at where AI genuinely excels and where it reliably fails, so you can stop fighting its limitations and start working with its strengths.

Part 2

What AI Actually Does (and Doesn't Do)

14 min read · Understanding AI

Part 2 of the "Build with AI" series

> **In short:** AI is exceptional at working with language at scale — summarizing, rewriting, translating, and extracting — and at recognizing patterns and synthesizing across huge bodies of text. It reliably fails at tasks needing real-time truth, exact calculation, or genuine judgment. Knowing where that line falls lets you stop fighting AI's limits and lean on its strengths.

There's a moment most people have had with AI — usually early on — where it does something that feels almost magical. It summarizes a dense 40-page report in 30 seconds. It rewrites a clunky paragraph into something crisp and clear. It explains a complex legal clause in plain English.

And then, sometimes in the very next session, it confidently tells you that a company was founded in 1987 when it was actually founded in 2003. Or it generates code that looks completely correct and breaks immediately when you run it. Or it gives you a list of academic citations that don't exist.

Both of these are the same tool. Neither is a glitch.

This is what makes AI genuinely confusing to work with: it operates at two extremes simultaneously — breathtakingly capable in some areas, surprisingly brittle in others — with no obvious signal telling you which mode you're in.

Most of the frustration people experience with AI comes from not knowing where the line is. They either trust it too much (and get burned by confident errors) or trust it too little (and leave enormous value on the table). Understanding where AI actually excels and where it reliably fails isn't just interesting — it's the foundational skill for getting real results.

What AI is exceptionally good at

1. Working with language at scale

This is AI's home territory. Summarizing, rewriting, translating, reformatting, extracting key points, changing tone, adapting content for different audiences — anything that involves transforming text from one form to another. AI does this faster and more consistently than almost any human, at any volume.

If you have 50 customer support emails and need to identify the top five themes, AI can do that in seconds. If you need the same piece of content rewritten for a technical audience and a non-technical one, AI handles both simultaneously. If you need a rough draft turned into polished copy, AI is an extraordinary first-pass editor.

Stop writing from scratch. Use AI to generate the first draft, then edit. Even a mediocre first draft is faster to fix than a blank page.

2. Pattern recognition and synthesis

AI has processed more text than any human could read in thousands of lifetimes. That gives it an unusual ability to spot patterns, make connections across domains, and synthesize information from multiple sources into a coherent whole.

Ask it to compare five different approaches to a problem. Ask it to identify what successful examples of something have in common. Ask it to find the structural tension in an argument. These synthesis tasks — where the value is in connecting dots across a large body of knowledge — are where AI genuinely earns its place.

Use AI for research synthesis. Give it multiple sources or perspectives and ask it to find the through-line, the contradictions, or the gaps.

3. Generating structured starting points

Blank page paralysis is real. AI is a remarkable antidote. Whether it's a project plan, a meeting agenda, a proposal outline, a set of interview questions, a framework for thinking about a decision — AI can produce a solid structured starting point in seconds that would take you 30 minutes to build from scratch.

The key word is "starting point." AI's first output on complex tasks is rarely the final answer. But it's almost always a useful scaffold.

When you're stuck on where to begin, ask AI for a structure first. "Give me an outline for X" before "write me X."

4. Explaining complex things simply

AI is a remarkably patient teacher. Ask it to explain a concept at different levels — "explain this like I'm a curious 12-year-old" versus "explain this assuming I have an MBA" — and it will calibrate accurately. Ask it to explain a technical concept using an analogy from cooking. Ask it to walk through a process step by step until you understand it. It doesn't get frustrated. It doesn't judge the question.

Use AI to get up to speed on unfamiliar domains before important meetings, decisions, or conversations. It's a better pre-briefing tool than most.

5. Processing data with clear, defined rules

Computers have always been good at following rules precisely and at scale — and AI inherits that strength, now with the ability to understand the context around the data too. Tasks with clear, defined logic are where AI is genuinely reliable: calculating tax scenarios, analyzing financial data, applying consistent formatting rules across thousands of rows, flagging entries that violate specific criteria, or running structured comparisons.

The key phrase is "clear rules." When the logic is unambiguous — if X then Y, always — AI handles it consistently and without fatigue. A tax calculation that would take an accountant an hour to verify across 500 line items takes AI seconds. A data quality check that requires applying the same rule to 10,000 records? Same story.

Any task where you can write down the rule clearly enough that a precise robot could follow it — that's a task AI can handle at scale. Think: data validation, formula application, structured classification, compliance checks.

6. Finding things in vast amounts of information

Humans are bad at searching through large bodies of text. We skim, miss things, get fatigued, and anchor on what we found first. AI doesn't have these limitations. Given a large document, a database of records, a long email thread, or a collection of reports, AI can locate specific information, surface relevant passages, identify what's missing, and spot inconsistencies — across the entire body of content, not just what catches the eye.

This is different from summarizing. Summarizing compresses. Finding surfaces specifics. "Does this 200-page contract contain any clause that limits liability in ways that conflict with our standard terms?" is a finding task — and AI handles it extraordinarily well.

Use AI as a search and retrieval layer over your own documents. Feed it a long contract, a research report, a transcript, or a collection of notes — and ask it to find, not just summarize.

Where AI still falls short — and what's changed

Before we go through the limitations, an important caveat: the boundary of what AI *can't* do is shifting fast.

Modern AI systems can now connect to external tools through frameworks like MCP (Model Context Protocol) — which lets AI call web search, query live databases, pull real-time financial data, read your files, and interact with external services. This isn't science fiction; it's how tools like Claude, ChatGPT, and others work today when properly configured.

What that means: several limitations that felt absolute a year ago are now soft limits or solvable with the right setup. We'll flag which ones have changed — and which ones haven't.

1. Real-time and specific factual information — *significantly improved with tools*

The classic AI limitation: it generates responses from training data, not live lookup. That meant wrong dates, fabricated statistics, outdated information — all delivered with full confidence.

This is still true of a base AI model used on its own. But connected to web search or live data sources via tools, AI can now retrieve accurate, current information before responding. Ask a tool-connected AI for today's exchange rate, a company's latest earnings, or a recent news event — and it can fetch the actual answer rather than guess.

Even with tools, AI can still hallucinate when it doesn't realize it needs to look something up, or when the information isn't available in the sources it can reach. The confidence problem hasn't fully disappeared.

For specific, verifiable facts — especially recent ones — use AI systems that have search or tool access enabled. And still verify anything consequential against a primary source.

2. Reasoning through novel, multi-step logic — *improved, but still different from human reasoning*

This one needs an honest update — because reasoning has been one of the fastest-moving areas in AI.

Modern "reasoning models" like OpenAI's o1/o3, Google's Gemini, and Claude's extended thinking mode use a technique called **chain of thought (CoT)** — they generate intermediate steps before arriving at an answer, essentially thinking out

loud. On math, logic, coding, and structured multi-step problems, this has been a genuine leap. These models handle complexity that would have stumped their predecessors entirely.

So the blanket statement "AI can't reason" is no longer accurate. For problems with recognizable patterns and defined structure, AI reasoning is now impressively reliable.

But several important gaps remain, and they're worth understanding because they're structural, not just a matter of more training data.

AI reasons in one direction. Humans backtrack. Human reasoning is recursive — we form a hypothesis, test it, feel something is off, backtrack, revise our assumptions, and try again, often without consciously deciding to. AI chain of thought is largely a one-pass forward generation. Each step is built on what came before. Even when it appears to "reconsider," it's generating the next most plausible token, not genuinely revisiting a prior belief the way a human does. This is why AI can go confidently down a wrong path and never course-correct.

AI also has no stakes in being right. Human reasoning is shaped by consequences. We reason more carefully when we know we'll be held accountable, when being wrong has real costs. AI has none of that friction — no fear of error, no ego investment. This makes it consistent, but it also means it can walk confidently into a wrong conclusion without any of the internal resistance a human would feel along the way.

Then there's confidence, which doesn't signal correctness. A reasoning model can produce a long, detailed chain of thought and arrive at a completely wrong conclusion — with the same confident tone it uses when it's entirely right. The length and apparent thoroughness of the reasoning doesn't reliably indicate whether the answer is correct. With a human expert, deep reasoning and high confidence are at least somewhat correlated. With AI, they're not.

Finally, genuinely novel situations without prior patterns still trip it up. AI reasoning shines on problems that resemble things it has seen before. Unusual edge cases, decisions that require reasoning from physical intuition, social dynamics with no clear precedent — these are where it still struggles. Human reasoning is embodied and adaptive. AI reasoning is pattern-completion, and when there's no pattern to complete from, the scaffolding gets shaky.

For structured, well-defined problems, use reasoning models freely — they're genuinely good. For novel, high-stakes, or consequential reasoning, treat AI as a thought partner that surfaces considerations, not a final authority. Always sanity-check the logic yourself. Pay special attention when AI sounds most confident: that's precisely when the "confidently wrong" failure mode is hardest to catch.

3. Knowing what it doesn't know — *still the most dangerous limitation*

AI doesn't have a reliable internal signal for uncertainty. A human expert asked something outside their knowledge will typically hesitate or say "I'm not sure." AI often doesn't. It fills the gap with plausible-sounding content — sometimes entirely fabricated — delivered in the same confident tone it uses when it's completely correct.

This is what the AI community calls "hallucination." Tool access reduces it on factual questions because AI can look things up. But it doesn't eliminate the underlying tendency, especially on niche topics, complex specifics, or questions at the edge of what any source can answer.

The more specific and niche the question, the more skeptical you should be of a confident answer. Treat AI output in unfamiliar territory as a starting hypothesis, not a conclusion.

4. Judgment that requires real-world stakes — *tools don't help here*

This one hasn't changed, and tools don't change it. AI has no skin in the game. It hasn't run a company, navigated a difficult client relationship, or felt the weight of a decision where being wrong has real consequences. It can surface frameworks, list considerations, and play devil's advocate — but it can't replace the judgment of someone who has actually been in the room.

"Should I fire this person?" "Is this partnership worth pursuing?" "Should I take this investor's money?" These aren't information problems. They're judgment calls that require lived context, relationship history, and real accountability.

Use AI to surface considerations you might have missed, not to make the call. "What are the arguments for and against?" is a better question than "what should I do?"

The practical framework: trust tiers

Once you understand these patterns, you can develop a working intuition for when to trust AI's output and when to verify it.

For language transformation — summarizing, rewriting, translating — brainstorming, generating structures and outlines, explaining concepts, drafting communication: trust freely. The cost of a small error here is low, and the efficiency gain is high.

For research synthesis, strategic frameworks, technical explanations, and code generation: trust but verify. AI is genuinely useful here, but always review the output before acting on it.

For specific facts, statistics, citations, legal or medical details, precise historical information: use with caution. Treat AI output in these areas as a starting point for research, not a conclusion.

For final judgment on consequential decisions, complex novel reasoning where every step matters, or anything where being wrong has serious consequences: don't rely on it.

A quick test to run right now

Pick something you've used AI for recently, and ask yourself: which tier did that task fall into?

If you were asking AI to summarize or rewrite something — you were in safe territory.

If you were asking AI for specific facts or data — did you verify them? If not, go back and check.

If you were asking AI to help you make a consequential decision — did you treat its output as one input among many, or as the answer? If the latter, that's worth revisiting.

Most people, once they think about it, realize they've been either over-trusting AI in tier 3 areas or under-using it in tier 1 areas. The awareness alone shifts how you work with it.

What the limitations actually tell you

Here's the thing about AI's limitations: they don't diminish what it's capable of. They just define the game.

A calculator is extraordinarily useful — and you wouldn't trust it to write your strategy memo. A GPS is invaluable — and you still need to know when the route it suggests doesn't make sense. AI is the same. Understanding what it does well and what it doesn't isn't pessimism. It's the prerequisite for using it well.

The people getting extraordinary results from AI aren't ignoring its weaknesses. They've learned to route the right tasks to it and keep judgment where it belongs —

with them.

What to remember from this post

AI operates at two extremes with no obvious warning signal. Knowing which mode you're in is the skill worth developing.

AI's home territory is language transformation: summarizing, rewriting, reformatting, explaining, synthesizing — anything that turns text from one form into another. AI doesn't know what it doesn't know, and it produces wrong facts with the same confident tone as correct ones. The more specific the question, the more skeptical you should be.

Use the trust tier framework as your default: trust freely for language work, verify for synthesis and code, stay cautious with specific facts, and keep consequential judgment to yourself. The best AI users aren't the most optimistic ones — they're the ones who've learned exactly where to deploy it and where to stay in the driver's seat.

Want the full framework?

This post covers how AI actually works — the honest version. The *AI Development Guide* by Jaehee Song builds on this foundation to show you how to apply these principles across the full arc of building with AI: from prompting and data to agents, vibe coding, and production deployment.

🛒 Amazon 📱 Apple Books ▶️ Google Play Books 🌐 All Platforms (Books2Read)

Next in the series: "The AI Lego Stack" — how modern AI tools fit together, and how to pick the right combination for what you're trying to build.

Part 3

The AI Lego Stack

10 min read · Understanding AI

Part 3 of the "Build with AI" series

> **In short:** Every AI solution is built from three layers: the **model** (the brain — Claude, ChatGPT, Gemini), **orchestration** (how tasks get coordinated and automated — n8n, agents), and the **interface** (how humans use it — chat, apps, editors like Cursor). Knowing which layer a tool belongs to ends the confusion of tab-hopping between dozens of AI products.

Here's a conversation that happens constantly right now.

Someone decides they want to "use AI" for their business. They open ChatGPT. They try a few things. Some of it works, some doesn't. They hear about Claude, switch over. Then someone mentions Cursor for coding, so they install that. Then a colleague says they should be using n8n for automation. Then there's Midjourney for images. Then Make.com. Then someone posts about a new tool on LinkedIn and suddenly there are seventeen browser tabs open and a creeping sense that the whole thing is too complicated to actually figure out.

Sound familiar?

The problem isn't the tools. The problem is that nobody explained how they fit together.

Once you see the structure — what each layer does, how they connect, what to pick for what job — the whole thing stops feeling overwhelming and starts feeling like a set of building blocks. Lego, not chaos.

The three layers of the modern AI stack

Think of every AI-powered solution as being built from three distinct layers, stacked on top of each other. Each layer has a job. Each depends on the one below it.

```
┌──────────────────────────────────┐ |        LAYER 3: INTERFACE      |  ← How h
```

Let's walk through each one.

Layer 1: The model — the brain

This is the AI itself. The large language model (LLM) that actually understands language, reasons, generates text, writes code, summarizes documents, answers questions.

The major players you need to know:

OpenAI — GPT-5.2 The current flagship, built for coding and agentic tasks across industries. GPT-5 mini and GPT-5 nano offer lighter, faster, and cheaper alternatives for well-defined tasks. Accessible through ChatGPT or the API.

Anthropic — Claude Opus 4.6, Claude Sonnet 4.6 Claude Opus 4.6 is Anthropic's most capable model — exceptional at long documents, nuanced writing, complex instruction-following, and extended reasoning, with a 1 million token context window. Claude Sonnet 4.6 is the everyday sweet spot: fast, highly capable, and cost-efficient for most use cases. Available through claude.ai or the API.

Google — Gemini 3.1 Pro / Gemini 2.5 Pro Gemini 3.1 Pro is Google's newest and most capable model (currently in preview), built for complex problem-solving and agentic tasks. Gemini 2.5 Pro is the current stable flagship — strong reasoning, very long context window, and deep integration with Google's ecosystem (Docs, Sheets, Gmail). Strong multimodal capabilities — handles text, images, audio, and video natively.

Meta — Llama 4 (open-source) Meta's latest open-source model family. Free, can be run locally or self-hosted. Important if you need data privacy, cost control, or customization that proprietary models don't allow. Llama 4 has closed the gap significantly with the leading proprietary models.

xAI — Grok 4.20 xAI's newest flagship, with industry-leading speed and agentic tool-calling capabilities. Real-time access to X (Twitter) data via X Search is its distinctive edge — useful for social listening, trend analysis, and anything where live public conversation is the signal.

The key insight: you don't need to pick one and stick with it forever. Different models have different strengths. Many builders use different models for different tasks — Claude Opus 4.6 for deep reasoning and long documents, GPT-5.2 for coding and agentic tasks, Gemini 3.1 Pro for Google-ecosystem workflows. The model is

interchangeable. Your prompts, your logic, your workflows — those are the assets worth investing in.

Layer 2: Orchestration — the coordinator

A model on its own is powerful but limited. It answers one question at a time. It can't automatically check your email every morning, pull data from your CRM, process it, and send a summary to Slack. It can't run a multi-step workflow without someone sitting there typing each prompt manually.

That's what the orchestration layer does. It coordinates: triggering the AI at the right time, feeding it the right data, connecting it to other tools, and acting on the output automatically.

This is the layer most people skip — and it's where the real leverage lives.

n8n — A visual workflow automation tool. You connect nodes (triggers, actions, AI calls) on a canvas. When X happens, do Y, then call AI, then do Z. Open-source, self-hostable, excellent for builders who want control. This is what powers most serious AI automation workflows.

Make.com — Similar to n8n but more beginner-friendly, cloud-based, with a large library of pre-built connectors. If you want to get something running fast without much technical setup, Make is your starting point.

LangChain / LlamaIndex — Developer-focused frameworks for building more complex AI pipelines in code. If you're working with a developer or learning to code, these give you fine-grained control over how AI interacts with data and tools.

AI Agents — Worth mentioning separately because they're increasingly their own orchestration layer. An agent is an AI that can plan a sequence of steps, use tools, and complete a goal with minimal human intervention — essentially a workflow that figures out its own steps. We'll dedicate a full post to agents later in this series.

Layer 3: Interface — the front door

This is what the user actually sees and touches. The interface is how humans interact with whatever you've built.

Chat interfaces — The simplest. A text box, a response. Claude.ai, ChatGPT, custom chatbots built on top of an API. Good for open-ended interaction.

Web apps and dashboards — Built with tools like Bolt, Lovable, Vercel v0, or traditional development. A proper UI that feels like a real product, not a chat window.

Embedded AI — AI built into an existing tool or workflow the user already uses. A button in Google Docs that summarizes the document. A Slack command that runs a report. An email that triggers an automated analysis. The user never thinks "I'm using AI" — they just use the tool.

Voice interfaces — Growing fast. AI that listens and responds. Customer service bots, voice assistants, meeting transcription and summarization tools.

The interface is often the last thing to build, but it's the first thing users judge. A brilliant AI workflow with a clunky interface will be abandoned. A simple workflow with a clean, intuitive interface will be used every day. Don't underestimate this layer.

How the layers connect: a real example

Let's make this concrete. Imagine you want to build a tool that monitors your competitors' websites, summarizes what changed, and sends you a weekly digest.

Layer 1 (Model): Claude Sonnet 4.6 or GPT-5.2 — reads the competitor content and generates a clear, concise summary of what's new and why it matters.

Layer 2 (Orchestration): n8n — runs every Monday morning, fetches the competitor URLs, passes the content to the AI model, formats the output, and sends it to your email or Slack.

Layer 3 (Interface): Your email inbox or a Slack channel — that's where the digest lands. You don't need a fancy UI; the interface is wherever you already spend your time.

Total coding required to build this: zero, if you use n8n's visual interface and a pre-built email connector. The logic is yours. The AI does the reading and summarizing. The orchestration does the scheduling and routing. The interface is already built — it's your inbox.

This is the Lego model in action. Each piece has a job. You assemble them. You don't build the bricks from scratch.

How to pick your stack

You don't need to master all of this at once. Here's a practical starting framework based on where you are:

If you're just getting started:

- Model: ChatGPT or Claude (use the chat interface directly)
- Orchestration: none yet — do things manually to understand the workflow first
- Interface: the chat window

If you want to automate something:

- Model: Claude Sonnet 4.6 or GPT-5.2 via API
- Orchestration: Make.com (easier) or n8n (more powerful)
- Interface: email, Slack, or a simple web form

If you want to build a real product:

- Model: Claude Opus 4.6 or GPT-5.2 via API, potentially multiple models for different tasks
- Orchestration: n8n or LangChain, with AI agents for complex workflows
- Interface: Bolt, Lovable, or v0 for rapid UI building without deep coding

Start with the simplest version that does the job. Add layers as the need becomes clear. Most people over-engineer the stack before they understand the problem. Understand the problem first (back to Post 1), then choose the stack.

The mistake most people make

They pick tools before they understand the problem.

They hear about n8n and build an automation before knowing what they want to automate. They set up LangChain before knowing if they need that level of complexity. They choose a model based on what's trending, not what fits the task.

The stack should follow the use case, not the other way around.

Before choosing any tool, ask: **What is the job to be done? Where does AI add value in that job? How will the result reach the person who needs it?** Those three questions map directly to Layer 1, Layer 2, and Layer 3. Answer them first. Then choose your Lego pieces.

What to remember from this post

Every AI solution has three layers — Model, Orchestration, Interface — and understanding which layer you're working in cuts through the tool overwhelm immediately. A model alone answers questions; orchestration makes it act automatically; interface makes it usable. You need all three for anything real.

The orchestration layer is where most of the leverage lives, and most people skip it. Connecting AI to triggers, data sources, and actions via tools like n8n or Make.com is what turns a chat interaction into a working system. Don't over-commit to one model — your prompts, logic, and workflows are the real assets. The model is a replaceable component. Start simple, add layers as needed. Complexity is a cost, not a feature.

Want the full framework?

This post covers the architecture. The *AI Development Guide* by Jaehee Song goes deeper into how each layer works in practice — with real examples of stacks built for specific use cases, from solo operators to enterprise workflows.

🛒 Amazon 📱 Apple Books ▶️ Google Play Books 🌐 All Platforms (Books2Read)

Next in the series: "Your Data is the Real Bottleneck" — why the quality of what you feed AI matters more than which AI you use, and how to know if your data is actually ready.

Part 4

Your Data Is the Real Bottleneck

9 min read · Data & Prompts

Part 4 of the "Build with AI" series

> **In short:** With AI, your data matters more than your model. "Garbage in, garbage out" still rules: feed AI inconsistent, incomplete, or messy information and it returns confident, plausible output built on shaky foundations. Choosing the right model is roughly a 10% problem — getting your data right is the 90% problem most people overlook.

Here's a scenario that plays out constantly.

A team spends weeks evaluating AI models. They test GPT-5.2 against Claude Opus 4.6. They debate which one handles their use case better. They read benchmarks. They run trials. They finally commit to a model and build their workflow.

Then they feed it their actual data — the spreadsheets, the CRM exports, the customer notes, the internal documents — and the results are a mess. Inconsistent. Unreliable. Sometimes useful, often wrong.

They blame the model. Switch to a different one. Same results.

The model was never the problem.

What's actually going on

There's a principle in data engineering that predates AI by decades: **garbage in, garbage out.** Feed a system bad data, and it produces bad results — regardless of how sophisticated the system is. This was true for databases in the 1980s. It's true for machine learning models. And it's especially true for AI today.

The difference is that AI is unusually good at hiding this problem. A language model can take inconsistent, messy, incomplete data and produce something that *sounds* coherent and confident. It fills gaps, smooths over contradictions, generates

plausible-sounding outputs. Which means you can spend a long time thinking the AI is doing fine — until you try to act on its outputs and realize they were built on shaky foundations.

Most people evaluating AI tools are evaluating the model. They should be evaluating their data.

What "data" actually means here

When we talk about data in the context of AI, we don't just mean spreadsheets and databases. We mean any information you feed into an AI to help it do its job. That includes:

- **Structured data** — spreadsheets, CSV files, database exports, CRM records
- **Unstructured text** — emails, meeting notes, customer feedback, internal documents, PDFs
- **Context you provide in prompts** — background information, instructions, examples
- **External sources** — websites, APIs, documents you retrieve and pass to the model

All of it is data. And all of it has quality issues that affect what AI can do with it.

The four data problems that kill AI projects

1. Inconsistency

This is the most common and most damaging. The same thing described in five different ways: "New York", "NY", "NYC", "New York City", "new york". A customer status that's "Active" in one system and "active" in another. A product name with a space in some records and a hyphen in others.

Humans handle inconsistency automatically — our brains normalize it without effort. AI doesn't. It treats "Active" and "active" as potentially different things. It notices the pattern variations and either gets confused or makes assumptions that compound errors downstream.

Pick any key field in your data — a category, a status, a name — and count how many variations of the same value exist. If you find more than two or three, you have an inconsistency problem.

2. Incompleteness

Missing fields. Empty cells. "N/A" where there should be a value. Records that are half-filled because the person entering them was in a hurry, or because the system didn't require it.

AI can work with incomplete data — but it will fill the gaps with assumptions. Sometimes those assumptions are reasonable. Often they're not. And you won't always know which is which.

The danger compounds when incompleteness is *systematic* — when the same fields are always missing for the same type of record. That means the AI is making the same wrong assumption over and over, consistently, at scale.

Check what percentage of your records have all key fields populated. If it's below 80%, your AI will be operating on assumptions more than facts.

3. Staleness

Data has an expiry date. A customer record from two years ago may have a wrong address, an old job title, a phone number that no longer works. A product description written in 2022 may describe features that no longer exist. A market analysis from 18 months ago is describing a different market.

When you feed AI stale data and ask it to do something current — make a recommendation, generate a proposal, summarize a customer's situation — it will work from that stale information and produce output that sounds authoritative but is built on an outdated picture of reality.

Ask yourself: when was the last time each major data source was verified or updated? If you don't know, that's the answer.

4. Context collapse

This is the subtlest problem. Your data makes perfect sense to you — because you carry years of context that the data doesn't capture.

"Customer is on hold" means something specific to your team: they requested a pause in service, they're waiting for a hardware shipment, they're mid-negotiation on a contract renewal. But that phrase alone, fed to an AI, could mean any of a dozen things. The AI doesn't have the organizational context to interpret it correctly.

This is especially true of shorthand, internal jargon, abbreviations, and codes that make perfect sense to insiders but are opaque to any outside reader — including AI.

Take a random sample of your records and ask someone unfamiliar with your business to interpret them. Every place they're confused is a context collapse risk.

The data readiness checklist

Before you build any AI workflow on top of your data, run through this checklist:

Consistency

- [ ] Key categorical fields use standardized values (not multiple variants of the same thing)
- [ ] Names, IDs, and references match across systems
- [ ] Date formats are consistent
- [ ] Text fields don't contain structured data that should be in separate fields

Completeness

- [ ] Key fields have >80% population rate
- [ ] Missing values are explicitly marked (not empty, not "N/A", not "unknown" inconsistently)
- [ ] No required fields are routinely left blank

Freshness

- [ ] You know when each data source was last verified
- [ ] Stale records are flagged or archived, not mixed with current ones
- [ ] Time-sensitive fields (contact info, prices, statuses) have a review cadence

Context

- [ ] Abbreviations and internal jargon are documented somewhere
- [ ] Status codes and categories have definitions
- [ ] Records can be understood by someone without internal organizational context

If you can check everything on this list, your data is AI-ready. If you're checking fewer than half, your AI results will be unreliable regardless of which model you use.

What to do about it

The good news: AI itself is an excellent tool for cleaning and preparing data. The process is:

Step 1: Audit first, fix second. Don't start cleaning data randomly. Use AI to help you audit — feed it a sample and ask it to identify inconsistencies, missing patterns, and ambiguous values. Let it surface the problems before you decide how to fix them.

Step 2: Standardize the high-impact fields first. You don't need to fix everything. Focus on the fields that your AI workflow will actually use. If you're building a customer outreach tool, customer name, status, and contact info matter more than internal notes. Fix the critical path, not the entire dataset.

Step 3: Document what you've standardized. A short reference document — what values are allowed in each field, what each status means, what the abbreviations stand for — is worth more than hours of cleaning. It prevents the problem from recurring and gives AI a reference to work from.

Step 4: Build freshness into the process. Data quality is not a one-time project. It decays. Build a simple review cadence — quarterly for slow-changing data, monthly for fast-changing — so you're not solving the same staleness problem repeatedly.

What this is really about

Choosing the right AI model is a 10% problem. Getting your data right is a 90% problem.

This isn't a knock on AI. It's a reflection of where the real work is in any information-intensive task. The model is a tool. The data is the raw material. No craftsperson blames their tools when they're working with bad material.

The builders who get consistently good results from AI aren't necessarily using the best models. They're using clean, consistent, well-documented data — and they've built habits to keep it that way.

That's the unsexy truth the AI demo world never shows you. The demos use pristine, pre-cleaned example data. Your reality is messier. And fixing that mess is the highest-leverage thing you can do before adding AI to anything.

Key takeaways

AI can produce confident, fluent-sounding outputs from bad data — which makes the problem harder to detect, not easier. Most data quality problems fall into one of four categories: inconsistency, incompleteness, staleness, and context collapse. Worth knowing which ones you have before you build. The model choice matters far less

than people think; data quality matters far more. Before you clean, use AI to identify what needs cleaning — feed it a sample and ask it to surface problems. And data quality is a habit, not a project. It decays. Build review cadences, document standards, and treat it as ongoing maintenance.

Want the full framework?

This post covers the data foundation. The *AI Development Guide* by Jaehee Song goes deeper — into how to structure your data for specific AI use cases, how to use AI to clean and enrich what you have, and how data quality connects to every layer of the AI stack.

🛒 Amazon 📱 Apple Books ▶️ Google Play Books 🌐 All Platforms (Books2Read)

Next in the series: "The Real Skill Isn't Coding — It's Defining the Problem" — why in an age of tools that can build almost anything, the bottleneck has shifted entirely to how precisely you can articulate what you want.

Part 5

The Real Skill Isn't Coding

10 min read · Data & Prompts

Part 5 of the "Build with AI" series

> **In short:** The real skill in building with AI isn't coding — it's defining precisely what you want. Vague inputs produce confident, well-structured answers to slightly the wrong problem, and the mistake only surfaces after you've built on it. Ten minutes spent sharpening the problem definition saves hours of rebuilding later.

Something remarkable has happened in the last two years.

The tools that build software have become so capable that the act of building itself is no longer the hard part. Cursor writes the code. Bolt scaffolds the app. Lovable generates the UI. Claude Opus 4.6 reasons through the architecture. You describe what you want, and something functional appears — often in minutes.

This should be liberating. And for many people, it is.

But there's a problem hiding inside this progress. As the cost of building dropped to near zero, something else became the bottleneck — something that was always there, but easy to ignore when building was hard.

The bottleneck is now: can you clearly define what you want to build?

Not technically. Not in code. In plain language. With enough precision that a powerful tool — or a smart person who just joined your team — could act on it without having to ask twenty clarifying questions.

That skill is harder than it sounds. And it separates the people who get extraordinary results from AI from the people who keep getting frustrating ones.

Why vague inputs produce vague outputs

There's a direct relationship between the precision of your problem definition and the quality of what AI builds for you.

This isn't unique to AI. It's true of every creative or technical collaboration. If you hire a designer and say "make it look good," you'll get their interpretation of good, not yours. If you brief a developer with "build something that handles orders," you'll get a system that handles orders in ways you never anticipated — and probably doesn't handle them in ways you assumed were obvious.

AI amplifies this dynamic. A language model is extraordinarily capable at filling in gaps — but the gaps it fills are based on pattern matching across everything it's been trained on, not on your specific context, your users, your constraints, your definition of success.

When you give AI a vague problem, it gives you a confident, well-structured answer to a problem that's slightly different from the one you actually have.

And here's the trap: it looks right. It sounds right. It passes a surface inspection. You start building on it — and three steps later you realize the foundation was off.

What a well-defined problem actually looks like

Most people think defining a problem means writing a longer description. It doesn't. Length is not precision.

A well-defined problem has four components:

1. The specific situation — not "I want to automate my sales process" but "I want to automatically send a follow-up email 48 hours after a sales call if the prospect hasn't responded, personalized based on what we discussed."

2. The specific user — not "customers" but "small business owners who signed up in the last 30 days and haven't yet connected their first data source."

3. The specific constraint — what's the boundary? What can't it do? What must it always do? "It should never send more than one follow-up per week" is a constraint. "It should feel personal, not automated" is a constraint.

4. The specific success condition — how will you know it worked? "The response rate on follow-ups increases" is a success condition. "It saves me two hours a week" is a success condition. "It doesn't embarrass us" is a success condition.

Without all four, you have a direction, not a definition. And a direction is not enough to build on.

The before and after: same goal, different results

Let's make this concrete with a real example.

Vague version: *"Build me a tool that helps my team manage customer complaints."*

This will produce something. It might even look impressive. But it will almost certainly miss the real problem because "manage customer complaints" could mean: a ticketing system, an AI responder, a categorization tool, a dashboard, an escalation workflow, or twenty other things.

Precise version: *"We receive about 50 customer complaints per week via email. Currently my team manually reads each one, decides if it needs a human response or just an acknowledgment, and assigns it to the right person. This takes about 3 hours a day. I want a tool that reads incoming complaint emails, classifies them by urgency (urgent / standard / FYI), drafts a first-response email for human review, and routes it to the right team member based on category. It should never send anything automatically — everything goes through human approval before sending."*

Same goal. Completely different brief. The second one could be handed to a developer, an AI agent, or a non-technical builder with Bolt or n8n — and all of them would produce something that actually solves the problem.

The difference isn't technical skill. It's thinking skill.

Why this is hard (and why most people skip it)

Defining a problem precisely is uncomfortable work. It forces you to make decisions before you know all the answers. It surfaces assumptions you didn't realize you were making. It reveals that the problem you thought you had might not be the actual problem.

Most people skip it because of a few patterns that feel intuitive but work against you.

Building feels like progress. Defining feels like delay. The moment you open Cursor or Lovable and start describing what you want, something is happening. The screen fills up. It feels productive. Spending thirty minutes writing a problem definition before touching any tool feels like wasted time — especially when the tools are so fast. This is exactly backwards. Ten minutes of clear problem definition saves hours of iteration on the wrong thing. The tool builds fast; the cost is reckoning with what you built, realizing it's not quite right, and rebuilding.

Vagueness also feels safer. A vague brief leaves room for the result to be interpreted generously. If you're not specific, you can't be specifically wrong. Precision requires

commitment — and commitment requires confidence in what you actually want.

And sometimes the real problem is uncomfortable. The process of defining a problem precisely can reveal that the real issue isn't what you thought. "We need a better complaint management tool" sometimes means "we have too few support staff." "We need to automate our reporting" sometimes means "we don't actually know what we're trying to measure." Precise problem definition can surface organizational truths that are more uncomfortable to face than a technical problem.

The problem definition framework

Before you open any AI tool, answer these five questions. Write the answers down — don't keep them in your head.

1. What is the current situation? Describe what's happening now in specific, concrete terms. Numbers where possible. Who does what, how often, how long does it take?

2. What is the pain? What specifically goes wrong, takes too long, costs too much, or doesn't happen that should? Not "it's inefficient" — what is the specific inefficiency?

3. Who experiences this? Specific role, specific context. Not "users" — which users, doing what, when?

4. What does the ideal outcome look like? If this is solved perfectly, describe a specific scenario. Walk through it step by step. What happens that doesn't happen now?

5. What are the hard constraints? What can't change? What must the solution always do or never do? What would make a technically correct solution still unacceptable?

When you can answer all five clearly, you have a problem definition. That definition is your brief. Feed it to Claude Opus 4.6, Cursor, or Bolt — and watch how dramatically the quality of what comes back changes.

The tool is not the strategy

Here's what's underneath all of this.

In an era where Cursor, Claude, Bolt, and Lovable can build almost anything you can describe, the competitive advantage has shifted entirely to description quality. To thinking quality. To the precision of your understanding of the problem you're solving.

This means the most important AI skill in 2026 is not prompt engineering. It's not knowing which model to use. It's not understanding APIs or automation tools.

It's the ability to think clearly about a problem before reaching for a tool.

That skill has always mattered. Every great product, every successful automation, every useful tool started with someone who understood the problem deeply before they tried to solve it.

What's changed is the amplification. When building was slow and expensive, a somewhat fuzzy problem definition still produced something useful — because the friction of the build process forced clarification along the way. Now the tool builds instantly. The fuzziness is preserved all the way through. You get a fast, well-executed answer to the wrong question.

The builders who win are the ones who slow down at the beginning to define precisely — so they can move fast with confidence through the build.

Your exercise: the one-paragraph brief

Take something you've been thinking about building or automating with AI. Something real — not hypothetical.

Write one paragraph that answers all five questions from the framework above. Be specific. Use numbers. Name the actual users. Name the actual constraint.

Then read it back. Ask yourself: could someone who has never met me, never seen my business, build exactly what I need from this description alone?

If the answer is no — keep refining. The work is in the refinement.

When the answer is yes — open the tool. You're ready.

Key takeaways

The bottleneck has shifted from building to defining. Tools can build almost anything you can describe — the constraint is now the quality of the description, not the technical execution. AI fills gaps confidently based on patterns, not your context, so a vague brief gets you a confident answer to the wrong question. A well-defined problem has four components: specific situation, specific user, specific constraint, specific success condition. Missing any one of them leaves meaningful gaps. Ten minutes of precise problem definition saves hours of rebuilding the wrong thing. And

the most important AI skill in 2026 isn't which model to use or how to write a prompt — it's the ability to think clearly about a problem before reaching for a tool.

Want the full framework?

This post covers the problem definition layer. The *AI Development Guide* by Jaehee Song goes deeper into how to translate sharp problem definitions into effective prompts, workflows, and complete AI solutions — with practical examples across different domains and use cases.

🛒 Amazon 📱 Apple Books ▶️ Google Play Books 🌐 All Platforms (Books2Read)

Next in the series: "Prompting is Programming" — once you have a sharp problem definition, how to translate it into prompts that consistently get the results you want.

Part 6

Prompting Is Programming

11 min read · Data & Prompts

Part 6 of the "Build with AI" series

> **In short:** Prompting is programming — you're giving a machine precise instructions, except a prompt is *interpreted* rather than executed literally, so every gap gets filled by the AI's best guess. Five patterns cover about 80% of effective prompting: role + task + context, format specification, constraints, chain of thought, and few-shot examples.

There's a persistent myth that prompting is just "talking to AI."

That if you type something naturally, in plain English, and the AI understands you, then you're already doing it right. That prompting is the easy part — the thing anyone can do without thinking about it.

This myth is costing people enormous amounts of time and producing mediocre results.

Prompting is not talking. Prompting is *instructing.* And like all forms of precise instruction — writing a legal contract, briefing a designer, writing a test case for a developer — the quality of the instruction determines the quality of the outcome.

The difference between a casual user of AI and a power user isn't the tool they use or the model they pick. It's the quality of their prompts. And prompt quality is a learnable skill — with a small set of patterns that, once you know them, change everything.

This post gives you those patterns.

Why prompting is programming

When a developer writes code, they're giving a machine precise, unambiguous instructions. Leave something vague and the code breaks, or worse, does something you didn't intend.

Prompting works the same way — with one important difference. Code is executed literally. A prompt is interpreted. The AI fills in gaps based on patterns — and those patterns may or may not match your intent.

This means:

- **Precision matters** — the more specific you are, the less the AI has to guess
- **Structure matters** — how you organize a prompt affects how the AI processes it
- **Context matters** — what you tell the AI before the task shapes how it approaches the task
- **Examples matter** — showing is more reliable than telling

Think of prompting as writing instructions for an extraordinarily capable intern who has read everything ever written, but knows nothing specific about you, your context, or what "good" looks like for your situation. Every gap you leave is a gap they'll fill with their best guess.

The 5 core prompt patterns

These are not tricks or hacks. They're structural patterns that work across almost every use case — writing, analysis, coding, research, decision support. Learn these five and you've covered 80% of what makes a prompt effective.

Pattern 1: Role + task + context

The single most impactful pattern. Tell the AI who to be, what to do, and what it needs to know.

Structure:

```
You are [role with specific expertise]. Your task is [specific action verb + del
```

Without the pattern: > "Write a LinkedIn post about our new product launch."

With the pattern: > "You are a B2B SaaS copywriter who specializes in writing for technical founders. > Your task is to write a LinkedIn post announcing our product launch that drives sign-ups for a free trial. > Context: Our product is an AI-powered requirements tool called Clearly (clearlyreqs.com). Our audience is product managers and startup founders who are frustrated with how long it takes to go from idea to

spec. The tone should be direct and confident — no hype, no buzzwords. Our typical customer is skeptical of marketing claims and responds to specificity."

The second prompt will produce something radically more useful. Not because it's longer — but because it eliminates three layers of guessing.

Pattern 2: Format specification

AI will default to a format that seems reasonable based on your prompt. That default is often wrong for your purpose. Specify the format explicitly.

Weak: > "Summarize this meeting transcript."

Strong: > "Summarize this meeting transcript. Format your response as: > - **Decisions made** (bullet list, 1 sentence each) > - **Action items** (bullet list, owner and deadline if mentioned) > - **Open questions** (bullet list, unresolved items that need follow-up) > Keep the entire summary under 200 words."

Format specification is especially important when:

- The output needs to go somewhere specific (an email, a slide, a form)
- You need to scan the output quickly
- The AI tends to be verbose when you need brevity (or vice versa)
- You're chaining AI outputs — the output of one prompt feeds the input of another

If you find yourself editing AI outputs to change their structure every time, that's a signal to add format specification to your prompt.

Pattern 3: Constraints and anti-patterns

Tell the AI what *not* to do. This is one of the most underused patterns — and one of the highest-leverage.

Every output you've received from AI that felt slightly off, slightly generic, or slightly wrong usually violated a constraint you had but didn't state.

Examples of constraints: > "Do not use bullet points — write in flowing prose." > "Do not mention competitors by name." > "Do not use the words 'innovative' or 'cutting-edge'." > "Do not assume the reader has technical background." > "Do not start any sentence with 'I'." > "Do not exceed 150 words."

Examples of anti-patterns (what to avoid doing): > "Avoid generic advice that could apply to any company — everything should be specific to our situation." > "Avoid restating the question before answering it." > "Avoid hedging language like 'it might be worth considering' — be direct and specific."

Constraints feel like restrictions, but they're actually a form of precision. They close the gap between "technically correct" and "actually what I wanted."

Pattern 4: Chain of thought (ask it to reason first)

For complex tasks — analysis, strategy, debugging, decisions — asking the AI to reason through a problem before giving you an answer dramatically improves output quality.

This works because it forces the model to build up to a conclusion rather than pattern-matching to a plausible-sounding answer. Think of it as asking someone to "show their work."

Without CoT: > "Should we expand to the Japanese market this year?"

With CoT: > "I'm considering expanding our B2B SaaS product to the Japanese market this year. Before giving me a recommendation, reason through the following: > 1. What factors typically determine success or failure for US SaaS companies entering Japan? > 2. What information would you need to make a confident recommendation? > 3. What are the strongest arguments for expanding now? > 4. What are the strongest arguments for waiting? > Then give me your recommendation based on that reasoning."

The second prompt will consistently produce more nuanced, more reliable output — because the model has to engage with the problem's complexity before it's allowed to give you an answer.

Use CoT for any decision with meaningful stakes, any analysis where the reasoning matters as much as the conclusion, debugging, and any time you've gotten a confident-sounding answer that turned out to be wrong.

Pattern 5: Examples (few-shot prompting)

The fastest way to close the gap between what you imagine and what AI produces: show it an example of what you want.

This is called "few-shot prompting" in technical literature, but the concept is simple: instead of describing what good looks like in words, demonstrate it.

Without examples: > "Write a product update email in our company's voice."

With examples: > "Write a product update email in our company's voice. > > Here are two examples of emails that match our tone: > > [Example 1 — paste a real email you've sent] > > [Example 2 — paste another real email] > > Notice: we keep sentences short, we use 'you' not 'users', we lead with the benefit not the feature, and we never use exclamation points."

The example does more work than a paragraph of description. It shows rhythm, vocabulary, structure, and tone — things that are very hard to describe but immediately clear when demonstrated.

Keep a small library of your best AI outputs. The next time you need something similar, use a previous output as a few-shot example rather than describing the style from scratch.

Putting it together: the full prompt

Here's what a well-constructed prompt looks like when all five patterns are applied:

Task: Write a cold outreach email to a potential partner.

```
You are a business development writer specializing in Korean-US technology partn

Your task is to write a cold outreach email to a director at a US smart city tec

Context:

• Seattle Partners is a US market entry firm focused on Korean technology compa

• We've worked with 20+ Korean startups across smart city, smart building, and

• The recipient is a Director of Business Development at a mid-size US smart ci

• They likely receive many generic partnership emails; ours needs to earn their

 Format:

• Subject line (under 8 words)

• Email body (under 150 words)
```

```
- Single clear call to action at the end

Constraints:
- Do not use the word "synergy" or "innovative"
- Do not be vague — mention at least one specific technology category we cover
- Do not start with "I hope this email finds you well" or any variation
- The tone should be direct and confident, not deferential

Here is an example of a previous outreach email that got a positive response: [
```

This prompt will consistently produce something you can actually use — because it eliminates almost all of the AI's guesswork about who you are, what you want, and what good looks like.

The iteration mindset

One more thing that separates casual users from power users: how they respond to imperfect outputs.

Casual users try one prompt, get something that's not quite right, and either accept it or start over from scratch.

Power users treat prompting as an iterative process. The first output reveals gaps. You tighten the prompt, add a constraint, give an example of what went wrong. The second output is better. You refine again. By the third or fourth iteration, you have something genuinely excellent — and more importantly, you have a prompt you can reuse.

The goal isn't to write a perfect prompt on the first try. The goal is to learn what your prompt was missing from each output, and add that precision to the next version.

Every prompt you refine is an asset. Save them. Build a library. That library is one of the most valuable things you can build for your workflow.

Key takeaways

The quality of your prompt determines the quality of your output — every time. It's a skill, not luck. Five patterns cover 80% of what makes a prompt effective: role + task + context, format specification, constraints and anti-patterns, chain of thought, and examples. Constraints are especially underused — telling AI what *not* to do closes the gap between technically correct and actually what you wanted. Showing the AI what you want works better than telling it, so keep a library of your best outputs and use them as few-shot examples. And every refined prompt you save is reusable. Over time, that library is one of the most productive things you can build.

Want the full framework?

This post covers the core patterns. The *AI Development Guide* by Jaehee Song goes deeper into advanced prompting for specific use cases — including how to prompt for agents, how to chain prompts in multi-step workflows, and how to adapt these patterns for different models and contexts.

🛒 Amazon 📱 Apple Books ▶️ Google Play Books 🌐 All Platforms (Books2Read)

Next in the series: "AI Agents — Build a Mini Workforce Without Writing Code" — how to go beyond single prompts into multi-step autonomous workflows that work while you sleep.

Part 7

AI Agents

17 min read · Building

Part 7 of the "Build with AI" series

> **In short:** An AI agent is a system that acts on its own: it has a **trigger** (it activates when something happens instead of waiting for a chat), uses **tools** (web, files, databases, APIs, code), and loops through plan–act–decide steps toward a goal. Agents come in three levels of autonomy — and you can build your first one with n8n.

So far in this series, we've talked about prompting — giving AI a task and getting a result. One prompt, one response. You ask, it answers.

That's useful. But it's also limited. Because most real work doesn't happen in a single exchange. It happens in sequences. Step A triggers Step B. The result of Step B feeds into Step C. Something external — an email, a form submission, a calendar event — kicks the whole thing off. And at the end, the result goes somewhere: an inbox, a spreadsheet, a Slack channel.

This is where AI agents come in.

An agent is not just an AI that answers questions. It's an AI that *acts* — that can plan a sequence of steps, use tools, make decisions, and complete a goal with minimal human involvement. It's the difference between a smart assistant who waits for instructions and one who can be handed a goal and trusted to figure out the steps.

And the remarkable thing about 2026: you can build agents without writing a single line of code.

What makes something an "agent"

The word "agent" gets thrown around a lot. Let's be precise.

A basic AI interaction looks like this: **You → Prompt → AI → Response → You**

An agent interaction looks like this: **Trigger → Agent → [Plan → Tool → Decision → Tool → Decision...] → Output → Destination**

The key differences:

Agents have triggers. They don't wait for you to open a chat window. They activate when something happens — a new email arrives, a form is submitted, a scheduled time occurs, a new row appears in a spreadsheet.

Agents use tools. They can search the web, read files, write to databases, send emails, call APIs, run code. They can reach beyond their context window into the real world.

Agents make decisions. At branching points — if this, then that — they evaluate the situation and choose a path. Not just generating text, but making conditional choices.

Agents have goals, not just tasks. A task is "summarize this document." A goal is "monitor our competitor's pricing page and alert me whenever something changes, with a summary of what changed and why it might matter."

Three levels of agents

Not every agent is the same. It helps to think in three levels:

Level 1 — Triggered automation A workflow that fires automatically when something happens, runs a fixed sequence of steps, and delivers a result. No real decision-making — just reliable, automatic execution of a defined process.

Example: Every time a new lead fills out your contact form, automatically look them up on LinkedIn, generate a personalized one-paragraph summary of who they are, and add it to your CRM notes before your team sees the lead.

Tools: n8n or Make.com, with AI for the generation step.

Level 2 — Decision-making agent A workflow that includes conditional logic — where the AI evaluates something and routes to different outcomes based on what it finds.

Example: When a customer support email arrives, the agent reads it, classifies it (billing issue / technical problem / feature request / complaint), checks if the customer is on a premium plan, and routes it accordingly — with different response templates for different combinations.

Tools: n8n with multiple branches, AI classification at decision points.

Level 3 — Goal-directed agent An agent that is given a goal rather than a script — and figures out its own steps to achieve it. It can plan, use tools in sequence, evaluate intermediate results, and adjust its approach.

Example: "Research the top 5 Korean smart city startups in the energy management space, compile their product descriptions, recent funding, and key differentiators, and produce a formatted comparison report."

The agent searches, reads, extracts, organizes, and writes — deciding which sources to use, when it has enough information, and how to structure the output.

Tools: Claude Opus 4.6 or GPT-5.2 with tool access, or platforms like Relevance AI or Claude Code for more complex orchestration.

Building your first agent: a real walkthrough

Let's build something real. We'll use n8n — the most powerful free/open-source automation tool available — to build a Level 1 agent.

The goal: Monitor a list of competitor websites. Every Monday morning, check if any of them have updated their pricing page, and if so, send a Slack message with a summary of what changed.

Step 1: The trigger In n8n, create a Schedule node. Set it to run every Monday at 8am. This is your trigger — the agent wakes up automatically without you doing anything.

Step 2: The data source Add an HTTP Request node for each competitor URL you want to monitor. This fetches the current content of their pricing page.

Step 3: The comparison Add a Function node that compares today's content against last week's stored version. If they match — stop. If they don't — continue to the next step.

Step 4: The AI analysis Add an AI node (n8n has native Claude and OpenAI integrations). Pass the old and new content with this prompt:

> "Compare these two versions of a competitor pricing page. Identify: (1) what changed, (2) whether prices went up or down, (3) any new plans or features added or removed, (4) your assessment of why they might have made this change. Be specific and concise."

Step 5: The output Add a Slack node. Send the AI's analysis to your #competitive-intel channel, formatted with the competitor name, date, and the summary.

Step 6: Store the new version Save today's content to compare against next week.

Total setup time: 45-60 minutes. Zero code. After that, it runs every Monday automatically — and you only see it when something actually changes.

That's an agent. It wakes up, checks things, decides whether to act, acts intelligently, and delivers the result where you need it.

What agents are best for

Not every task benefits from an agent. Here's where they genuinely shine:

Repetitive multi-step processes — anything you do the same way every time, with multiple steps, is a candidate for automation. The agent does it reliably, consistently, at any hour.

Monitoring and alerting — watching for changes, thresholds, or events across sources you can't manually check all the time. Competitor prices, news mentions, lead activity, system statuses.

Data enrichment pipelines — taking raw inputs (a new contact, a new lead, a new piece of content) and automatically enriching them with AI analysis before a human needs to touch them.

First-pass triage — handling the classification and routing of inbound requests (emails, support tickets, form submissions) before human review, so humans only deal with pre-qualified, pre-sorted work.

Report generation — pulling data from multiple sources on a schedule, running AI analysis, and delivering a formatted report without anyone having to compile it manually.

Where the field is moving right now

The single-agent use cases above are table stakes. The frontier in 2026 is moving in three directions worth knowing about — even if you're not ready to build them yet.

Agent swarms

A swarm is multiple agents working in parallel, each handling a different part of a larger task, coordinating their outputs toward a shared goal. Instead of one agent

doing everything sequentially, you have a team of specialized agents running simultaneously.

Think of it like a project team: one agent researches, another drafts, a third fact-checks, a fourth formats. They hand off to each other, compare results, and the orchestrating agent assembles the final output.

Frameworks like **OpenClaw** (an open-source multi-agent orchestration framework), **LangGraph**, and **CrewAI** are purpose-built for this pattern. CrewAI in particular has become popular for non-developers — you define each agent's role, tools, and goal in plain language, and the framework handles the coordination.

Real example: A market research swarm where Agent 1 searches for industry news, Agent 2 pulls competitor data, Agent 3 analyzes customer reviews, and Agent 4 synthesizes everything into a structured report — all running in parallel, completing in minutes what would take a human analyst hours.

AI employees

This is the concept generating the most conversation right now — and the most legitimate excitement. An AI employee is an agent given the same tools a real employee would have: a computer, a browser, access to company systems, a phone number, a payment method, email, calendar.

Companies like **Artisan**, **Relevance AI**, and **11x** are already selling "AI SDRs" (Sales Development Representatives) — agents that prospect leads, research them, write personalized outreach, follow up, handle objections, and book meetings. Fully automated, running 24/7.

The more striking development: some companies are now giving agents **real payment capabilities** through services like Stripe Agent Toolkit, allowing agents to process refunds, issue credits, and handle billing operations autonomously — within defined limits and with audit trails.

Phone-capable agents are also maturing fast. Tools like **Bland AI** and **Vapi** let you deploy agents that make and receive real phone calls, handle customer service conversations, qualify leads, and book appointments — indistinguishable from a human caller to most people.

The honest caveat: these are powerful but still early. They work well for narrowly defined, high-volume tasks with clear rules. They break on edge cases, nuance, and novel situations. The "AI employee" framing is slightly ahead of the reality — but the direction is clear and the pace is fast.

Computer-using agents

The newest frontier: agents that can literally use a computer the way a human does. They see the screen, move the mouse, click buttons, fill forms, navigate websites — without needing an API or integration.

Anthropic's Computer Use, **OpenAI's Operator**, and **Google's Project Mariner** are all exploring this. The implication is significant: any task a human can do on a computer becomes automatable, even if there's no API for it. Legacy software, government portals, any web interface — an agent can operate it.

This is still early and unreliable for production use. But it closes the last gap between "what an agent can access" and "what a human can access on a computer."

Memory and communication: how agents think across time

Once you understand agents, the natural next question is: do they remember anything? And if there are multiple agents working together, how do they talk to each other?

These are the right questions — and the answers are more nuanced than most introductions cover.

The four types of agent memory

In-context memory (short-term) This is the agent's active working memory — everything in its current context window. What it's seen, done, and said in the current session. Fast, immediately accessible, and completely gone when the session ends. Think of it like RAM. Powerful while running. Wiped when you turn it off.

The practical limit: even a 1M token context window fills up during long multi-step workflows. When it fills, older content gets dropped or summarized. Agents that don't manage this well start "forgetting" earlier steps.

External memory (long-term) Agents can read from and write to external storage — databases, files, spreadsheets, or specialized memory stores. This is persistent memory that survives between sessions. When an agent needs to remember something across runs — a customer's history, a previous decision, a fact from last week — it writes to external storage and retrieves it next time.

Tools like **Mem0** and **Zep** are built specifically for this, storing memories as searchable embeddings so agents retrieve *relevant* context rather than loading everything. This is what makes a "personal assistant" agent feel like it actually knows you over time.

Procedural memory (baked-in skills) This is the model's training — what it inherently knows how to do. Write code, summarize text, analyze data. You can't add to it at runtime; it's fixed per model version. But you can extend it by giving the agent tools and instructions that expand what it can act on.

Episodic memory (action history) A log of what the agent has done — past tool calls, past outputs, past decisions. Stored externally and retrieved at the start of a new session so the agent understands "what happened last time." Critical for avoiding duplicate actions and infinite loops.

How agents talk to each other

In a swarm or multi-agent system, agents need to coordinate. Three main patterns:

Shared memory (blackboard pattern) All agents read from and write to a shared data store. Agent 1 writes research findings to the blackboard. Agent 2 reads those and writes a draft. Agent 3 reads the draft and edits it. No direct agent-to-agent messaging — they coordinate through shared state. Simple, reliable, easy to debug. The preferred pattern for most production systems.

Direct messaging (message passing) Agents send structured messages directly to each other — like a Slack DM between teammates. Agent 1 sends its output to Agent 2 with explicit instructions for what to do with it. Frameworks like **CrewAI** and **AutoGen** use this model. Agents have defined roles and pass work through a message queue.

Orchestrator pattern A central "manager" agent controls the workflow. It assigns tasks to worker agents, collects their outputs, evaluates quality, and decides next steps — including sending work back for revision if it falls short. This mirrors how a good human manager runs a team. The orchestrator has the full picture; workers handle specialized subtasks. Most enterprise-grade agent systems use some form of this.

The memory problem that actually bites you

Here's what most people don't realize until they've built something: **agents don't automatically know what they've already done.**

In a multi-step workflow, if the agent doesn't explicitly record its progress, a failure halfway through means starting over from scratch. If it runs again next week, it has no idea it already processed something — and it will process it again.

This produces a category of bugs unique to agents:

- **Duplicate actions** — agent sends the same email twice because it never recorded sending it the first time

- **Lost context** — agent makes a decision without knowing a relevant prior decision from earlier in the workflow
- **Infinite loops** — agent keeps retrying something that failed, without any record that it already failed

The fix is explicit state management — writing checkpoints to external storage at every meaningful step, so the agent can always answer: *what have I done, what do I still need to do, what failed?* Every serious agent framework has a pattern for this. It's unglamorous. It's also what separates a reliable production agent from a demo that breaks in the real world.

The common mistakes

Automating a broken process An agent will faithfully execute a bad process at ten times the speed. Before you automate, make sure the underlying process is actually correct. Automation amplifies — both good and bad.

No human checkpoint on consequential outputs For anything that reaches a customer, a partner, or the outside world — always build in a human review step before sending. An agent that drafts is powerful. An agent that sends without review is a liability.

Over-engineering the first version Start with Level 1. A simple triggered automation that saves two hours a week is worth more than a complex Level 3 agent that takes weeks to build and breaks often. Get the simple version working, learn from it, then add complexity.

Not planning for failures External services go down. APIs change. Pages 404. Build error handling into every agent — what should happen when a step fails? At minimum, send yourself an alert so you know something broke.

Agents and MCP: the connective tissue

Underpinning all of the above — from basic Level 1 automations to full AI employee setups — is MCP (Model Context Protocol), which we first discussed in Post 2.

MCP is what lets agents connect to external services through standardized interfaces rather than fragile custom integrations. Your agent can natively access Google Drive, Gmail, Slack, GitHub, your CRM, your accounting tool — through a growing library of standardized connectors that any agent framework can use.

This is why agent swarms are becoming practical: when every agent speaks the same connection protocol, coordinating them across different services becomes dramatically simpler. And it's why the AI employee concept is accelerating — once an agent can connect to payment systems, phone APIs, and browser interfaces through MCP, the operational capabilities expand rapidly.

When you're evaluating any agent tool or platform, ask whether it supports MCP. Those that do will compound in capability over time as the connector library grows. Those that don't will eventually hit a ceiling.

Your first agent challenge

Don't start with something ambitious. Start with something annoying.

Think about something you do repeatedly that follows the same steps every time. Something that takes 15-30 minutes, happens at least weekly, and feels like it shouldn't require a human. A report you compile. An update you send. A check you run. A summary you write.

That's your first agent candidate.

Map out the steps on paper first. What triggers it? What are the sequential steps? Where does a human decision currently happen — and could a rule replace that decision? What's the output and where does it go?

Once you have that map, open n8n or Make.com and build it. Start with Level 1. Get it running. Then watch it run on its own for a month — and notice what you'd want to add.

Key takeaways

An agent is AI that acts, not just answers — triggers, tools, decisions, goals are what separate an agent from a chatbot. Start at Level 1: most of the value lives in simple triggered automations, and complexity comes later when you understand the problem deeply. n8n and Make.com let you build agents without code; most non-developers can get a working agent running in an afternoon. Automate the draft, the classification, the routing — but keep humans in the loop for anything that goes to the outside world. The best first agent is the one that eliminates a small, repeatable frustration. And without explicit checkpoints written to external storage, agents duplicate work, lose context, and loop on failures — state management is what separates a reliable agent from a flaky one.

Want the full framework?

This post covers the foundations of agents. The *AI Development Guide* by Jaehee Song goes deeper — into multi-agent architectures, how to design agent workflows for specific business domains, and how to combine agents with the prompting and data foundations we've covered in earlier posts.

🛒 Amazon 📱 Apple Books ▶️ Google Play Books 🌐 All Platforms (Books2Read)

Next in the series: "Vibe Coding — Build a Real App Using Plain English" — how to go from idea to a working, deployed application without writing a line of code yourself.

Part 8

Vibe Coding

11 min read · Building

Part 8 of the "Build with AI" series

> **In short:** Vibe coding is using AI — coding assistants and generative UI tools — to build real software from plain-language descriptions: you describe, the AI implements, you review and iterate. It works best not by winging it, but by being precise about what you want, systematic about testing, and disciplined about iteration through a repeatable workflow.

In 2025, "vibe coding" became the Collins Dictionary word of the year.

That's remarkable for a technical term — but it captures something real. The idea that you can describe what you want to build in plain language, and an AI will build it, has moved from science fiction to daily practice for millions of people in the span of about two years.

But here's what the hype doesn't tell you: most people who try vibe coding get frustrating results. Not because the tools don't work, but because they approach it the same way they approach prompting a chatbot. They describe what they want loosely, get something that looks almost right, patch it endlessly, and never quite get to something they'd show to anyone.

This post is about how to actually do it — the mindset, the workflow, and the practical steps to go from an idea to a working, deployable app without writing code yourself.

What vibe coding actually is

Vibe coding is using AI — specifically AI coding assistants and generative UI tools — to write the code for you, based on your natural language descriptions. You describe. The AI implements. You review, test, and iterate.

The name comes from the feeling of it: you get into a flow state where you're thinking about *what* to build and *why*, not *how* the code works. The implementation layer

almost disappears.

But "vibe" is a bit misleading. The best vibe coders aren't just winging it. They're precise about what they want, systematic about testing, and disciplined about iteration. The vibe is in the experience — not in the approach.

The term was coined by Andrej Karpathy, one of the founders of OpenAI and former AI director at Tesla. His description was precise: describe what you want, the AI generates code, you run it, describe what's wrong or what you want next, and iterate. You're not writing code. You're directing it.

What made this possible

Three things converged to make vibe coding genuinely viable.

Models got good enough at code. Today's frontier models — Claude Opus 4.6, GPT-5.2 — write production-quality code in dozens of languages, debug their own mistakes, and reason about architecture. They don't just autocomplete; they architect.

Context windows got large enough. An entire codebase can now fit in a single context window. The AI sees everything at once — your components, your logic, your styles — and makes changes coherent across the whole system.

Purpose-built tools emerged. Cursor, Bolt, Lovable, Replit, and v0 are not just text editors with AI bolted on. They're environments designed around the vibe coding workflow — where your description is the primary input and code is the output you never have to fully understand.

The combination means the gap between "I have an idea" and "here is a working thing" has collapsed from months to hours.

The tools in 2026

The field has matured significantly. Here's what's actually being used:

For full-stack web apps (no setup required):

- **Bolt.new** — describe your app, get a full-stack application. Strong for rapid prototyping. One of the most beginner-friendly options.
- **Lovable** — similar to Bolt with a polished UI focus. Particularly good for consumer-facing products where visual quality matters.

- **Vercel v0** — AI-powered component generation tightly integrated with the Vercel deployment ecosystem. Best if you're in the Next.js world.

For code-level building (more control):

- **Cursor** — VS Code fork with AI deeply integrated. You write prompts, Cursor writes, edits, and refactors code inline. The go-to for developers — but increasingly used by non-developers who want more control.
- **Claude Code** — Anthropic's agentic coding tool. Handles entire codebases, plans multi-file changes, and executes them. Particularly powerful for complex projects.
- **Windsurf** — a strong Cursor alternative with different strengths in agent-mode coding.

For specific use cases:

- **Replit** — browser-based, no local setup. Great for learning and quick experiments.
- **Glide / AppSheet** — data-driven apps and internal tools built directly on spreadsheets or databases.

The right choice: if you want something visible fast with no technical setup, use Bolt or Lovable. If you want more control and are willing to learn, try Cursor. For complex multi-file projects, Claude Code.

The vibe coding workflow

This is where most guides fall short. They show you the tool, not the process. Here's the actual workflow that produces real results:

Phase 1: define before you build

Go back to Post 5. Before you open any tool, write your one-paragraph problem definition. Know:

- What the app does (specifically)
- Who uses it (specifically)
- What a successful first version looks like (specifically)
- What it does *not* need to do yet

The MVP trap: most people try to build the full vision on their first attempt. Don't. Define the smallest version that would be genuinely useful. Everything else is version 2.

Phase 2: scaffold the structure first

Don't start by asking the AI to build the whole app. Start by asking it to plan.

> "I want to build [your one-paragraph description]. Before writing any code, give me: > 1. The key screens or pages this app needs > 2. The main data it needs to store and display > 3. The user flow from first open to completing the main action > 4. Any potential complexity I should think about before building"

Review this. Push back on anything wrong. Add anything missed. Five minutes here prevents hours of rebuilding.

Phase 3: build in layers

Build incrementally — one feature at a time, tested before moving to the next.

- **Layer 1: skeleton** — navigation, basic screens, placeholder content. No logic yet.
- **Layer 2: data** — what gets stored, how it's organized, basic display of real data.
- **Layer 3: core action** — the single most important thing the app does. Working end to end.
- **Layer 4: secondary features** — everything else, added one at a time.

Each layer gets confirmed before the next begins. This is what separates a working app from an impressive demo that breaks as soon as you click anything unexpected.

Phase 4: the debugging mindset

Things will break. This is not failure — it's the nature of software.

When something breaks, don't patch randomly. Instead:

Describe precisely: > "When I click Submit, nothing happens. I expected it to save the entry and show a success message. The button appears and is clickable but has no effect."

Show the exact error: copy it verbatim from the browser console. Don't paraphrase.

Ask it to explain before fixing: > "Before fixing this, explain what you think is causing it and why."

If the AI diagnoses incorrectly, you catch the misdiagnosis before a wrong fix makes things worse. And you learn something about how your code works.

Phase 5: test like a user, not a builder

Builders test happy paths. Users find edge cases.

After each feature:

- What happens if you leave a required field empty?
- What happens if you click the same button twice quickly?
- What happens on a mobile screen?
- What happens if there's no data yet?

These aren't obscure edge cases. They're things real users do in the first five minutes.

A real walkthrough: building a lead tracker

Here's how you'd build a simple lead tracking tool for a consulting firm — from nothing to deployed in an afternoon.

The brief: *A web app where our team can log new leads, record their stage (new / contacted / meeting scheduled / proposal sent / closed), add notes, and see all leads in a table sorted by stage.*

Step 1: Open Bolt.new

Step 2: structure prompt > "I want to build a simple lead tracker web app. Before coding, tell me: what screens does it need, what data should it store, and what's the simplest first version?"

Review the response. Confirm and proceed.

Step 3: scaffold prompt > "Build the skeleton: a main page with a table showing leads, an Add Lead button, and a simple form with fields for: company name, contact name, email, phone, status (dropdown: New / Contacted / Meeting Scheduled / Proposal Sent / Closed Won / Closed Lost), and notes. Use placeholder data for now. Make it clean and professional."

Does it look structurally right? Good — move on.

Step 4: wire the data > "Make the Add Lead form actually work — when submitted, the new lead should appear in the table. Store in app state for now."

Test. Add a lead. Does it appear? Yes → proceed. No → debug first.

Step 5: add filtering > "Add the ability to filter leads by status. When I click a status, only those leads show. Add a 'Show All' option."

Test the filter. Does it work correctly?

Step 6: deploy Bolt has a built-in deploy button. Click it. Live URL in seconds, shareable with your team immediately.

Total time: 2–3 hours. Zero code written by you. A working, deployed app.

The honest limits

Vibe coding is powerful, but you'll hit these walls and should know they're coming.

Complex business logic — simple CRUD apps work well. Apps with complex conditional logic, multi-step calculations, or intricate state management are harder to build and much harder to maintain when things go wrong.

Production security — apps for internal use or prototypes are fine. Apps handling sensitive user data, payments, or requiring proper authentication need a developer involved, or a managed security platform (Supabase Auth, Clerk, Firebase).

Database at scale — in-memory state is fine for prototypes. When you need real persistence or user accounts across sessions, wiring in a proper database is a meaningful step up in complexity.

Long-term maintenance — vibe-coded apps accumulate technical debt. After many rounds of iteration the code can become inconsistent and hard to extend. Keep apps focused. When they need significant growth, consider a clean rewrite.

Who this actually benefits

The deeper shift vibe coding represents isn't about replacing developers. It's about who gets to build.

Before: if you had a problem that needed a software solution, you hired a developer, used a tool that didn't quite fit, or waited.

After: you build the tool that fits your exact situation. The internal tool your team actually needs. The prototype that shows investors what you're building. The customer-facing feature you've been requesting for months. In an afternoon.

The people who get the most from this are domain experts — people who understand a problem deeply but couldn't previously translate that understanding into software. A consultant who knows exactly what clients need. An operations manager who sees the inefficiency every day. A founder with the vision but not yet a technical co-founder.

That's who this is for. And the tools are good enough now.

What to take from this

The tools are good. What separates working results from frustrating ones is discipline: define before building, build in layers, test as you go.

Start with the smallest useful version. Build the MVP. Everything else is version 2. Over-scoping the first version is the most common reason vibe coding projects stall.

Each layer confirmed before the next: skeleton, data, core action, secondary features. Debug precisely, not randomly — describe the problem clearly, show the exact error, ask the AI to explain before it fixes.

The real shift isn't replacing developers. It's letting people who understand problems deeply build solutions themselves.

> **Already shipped a demo?** The follow-up guide — Beyond Vibe Coding — picks up right here: how to take a working vibe-coded app from "it runs on my laptop" to a real, paying service, one practical stage at a time.

Want the full framework?

This post covers the vibe coding workflow. The *AI Development Guide* by Jaehee Song goes deeper — into how to maintain and extend vibe-coded apps, how to wire in databases and authentication, how to move from prototype to production, and how to work with developers when you hit the ceiling of no-code tools.

🛒 Amazon 📱 Apple Books ▶️ Google Play Books 🌐 All Platforms (Books2Read)

Next in the series: "AI Code Assistants — Your New Pair Programming Partner" — how developers and non-developers alike can use AI coding tools to go deeper than vibe coding when the project demands it.

Part 9

AI Code Assistants

16 min read · Building

Part 9 of the "Build with AI" series

> **In short:** AI code assistants — Cursor, Windsurf, GitHub Copilot, Claude Code — sit inside your code editor and collaborate at the code level, explaining, refactoring, debugging, generating, and reviewing while you stay in control. Unlike vibe-coding tools that hide the code, assistants keep it visible — so you can understand, maintain, and trust what gets built.

Post 8 covered vibe coding — the experience of describing what you want and watching an AI build it. For many use cases, that's enough. You get a working app in an afternoon without touching code directly.

But there's a ceiling.

When your project gets complex — when you're debugging something subtle, extending an existing codebase, optimizing for performance, or working on a team with shared standards — the pure "describe and receive" workflow starts to strain. You need more control, more visibility into what's actually happening, and a tighter feedback loop with the code itself.

This is where AI code assistants come in.

Not as a replacement for vibe coding — as its natural extension. When you're ready to get closer to the code without abandoning AI entirely, code assistants are the bridge. And in 2026, they've become so capable that even non-developers use them productively — not to write code from scratch, but to understand, extend, and maintain what they've built.

What's different about code assistants

Vibe coding tools (Bolt, Lovable, v0) generate entire applications from descriptions. They manage the architecture, the files, the deployment — you see very little of the

underlying code.

Code assistants work differently. They sit inside your development environment — your code editor — and collaborate with you at the code level. You see the code. You can edit it directly. The AI can explain, refactor, debug, generate, and review — but you remain in control of what actually gets written.

The distinction matters because:

- **Visibility** — you understand what's in your codebase, which means you can reason about it, spot problems, and make deliberate changes
- **Precision** — you can instruct the AI at the level of a specific function, a specific bug, a specific refactor rather than a whole feature
- **Maintainability** — code you understand is code you can maintain; code that appeared as if by magic is code that breaks mysteriously
- **Team compatibility** — when working with developers, you need to understand and modify shared code, not just generate new pieces

---

The main players in 2026

Cursor is the dominant code assistant as of 2026. It's a full VS Code fork — meaning it looks and works exactly like the most popular code editor in the world — with AI built deeply into every part of the experience. You can:

- Select any piece of code and ask the AI to explain it, refactor it, or find bugs in it
- Describe a change in natural language and have Cursor implement it across multiple files
- Use "Composer" to make large, coordinated changes across an entire codebase
- Chat with the AI about your entire project with full context of every file

Cursor's particular strength is **codebase-wide understanding**. It doesn't just see the file you're editing — it can see your entire project, understand how pieces relate to each other, and make changes that are coherent across the system.

Windsurf (by Codeium) is a strong Cursor alternative with slightly different strengths, particularly in its "Cascade" agentic mode, which can plan and execute multi-step coding tasks autonomously. Many developers switch between Cursor and Windsurf depending on the task.

GitHub Copilot (by Microsoft/GitHub) is the most widely deployed code assistant in enterprise settings. Its "Agent mode" now handles multi-file edits and whole-task

completion, not just line-by-line suggestions. If your team already uses GitHub, Copilot integrates naturally into that workflow.

Claude Code (by Anthropic) is a terminal-based agentic coding tool — the most capable for complex, multi-step, autonomous coding tasks. Unlike the editor-based tools, Claude Code operates in the command line and can plan entire sprints of work, execute them, test the results, and iterate. Less beginner-friendly, but extraordinarily powerful for the right tasks.

OpenAI Codex is OpenAI's cloud-based coding agent — distinct from the editor tools above. Rather than sitting inside your IDE, Codex operates entirely in the cloud. You assign it a task, it spins up a secure sandboxed environment preloaded with your GitHub repository, works autonomously (reading and editing files, running tests, checking types), and returns a completed pull request for your review. Task completion typically takes 1–30 minutes. It's powered by GPT-5-Codex, a version of GPT-5 optimized specifically for software engineering. The key use case: offloading well-scoped, repeatable tasks — refactoring, writing tests, fixing bugs, generating documentation — without breaking your focus. By March 2026 it had more than 2 million weekly active users, and is increasingly positioned as a broader enterprise agent platform. Available to ChatGPT Plus, Pro, Business, Enterprise, and Edu subscribers.

Google Antigravity is Google's agent-first IDE, announced in November 2025 and now one of the most talked-about tools in the space. Unlike Cursor or Copilot, which layered agent capabilities onto existing editor frameworks, Antigravity was designed from the ground up around multi-agent autonomous execution. It introduces a "Manager View" where you can dispatch multiple agents to work on different parts of your codebase simultaneously, each producing auditable Artifacts — task plans, implementation plans, screenshots, browser recordings — so you can review what happened and why. Its standout feature: a native browser agent that can autonomously test and validate web UIs without you switching windows. Antigravity supports Gemini 3.1 Pro, Claude Opus 4.6, Claude Sonnet 4.6, and GPT-OSS-120B, and is currently free in public preview with generous model quotas. Honest caveat: rate limits tightened after the initial preview honeymoon and some users have reported reliability issues — excellent for experimentation and solo development, less battle-tested for production team workflows.

JetBrains AI integrates into the JetBrains suite (IntelliJ, PyCharm, WebStorm) — relevant if your team uses JetBrains IDEs, which remain common in enterprise Java and Kotlin development.

Five ways non-developers use code assistants

You don't need to be a developer to get value from code assistants. Here are the five ways non-developers and vibe coders actually use them:

1. Understanding code you didn't write

Select any piece of code and ask: > "Explain what this code does in plain English. What does it take as input and what does it return? What would happen if I changed X?"

This is the most underrated use. When your vibe-coded app produces unexpected behavior, being able to read and understand the code — even imperfectly — is the difference between debugging effectively and patching blindly.

2. Extending existing functionality

Instead of rebuilding, extend. Select the relevant part of your existing code and prompt: > "I want to add a feature that lets users export this table as a CSV file. Looking at how the table is currently built, what's the best way to add this? Write the code."

The AI sees what already exists and generates code that works with it — rather than generating something that might conflict with your existing structure.

3. Debugging with context

When something breaks, paste the broken code and the error message and ask: > "This function is throwing this error: [paste error]. Here's the function: [paste code]. Explain what's causing this and show me the fix."

Unlike debugging through a no-code tool where you're one level removed, you get a precise explanation of the problem — not just a patch, but an understanding of what went wrong.

4. Code review and quality improvement

Select a piece of code and ask: > "Review this code. What are the potential problems? Is there a cleaner or more efficient way to write this? What edge cases might this miss?"

This is particularly valuable when you're about to show your code to a developer — it lets you clean up obvious issues before a real review.

5. Translating between formats

Code assistants are excellent at transformations: > "Convert this JavaScript function to Python." > "Take this SQL query and convert it to use our ORM's query builder syntax." > "This function takes an array as input — rewrite it to accept the same data as a JSON object instead."

These transformations are tedious for humans and trivial for AI.

The workflow that actually works

The single most important habit for getting good results from code assistants: **always show context, never describe in isolation.**

Bad prompt: > "Write a function that sends an email."

Good prompt: > "Here is my existing notification service: [paste code]. I need to add a function that sends an email when a user's subscription expires. It should use the same email client we're already importing at the top of this file, and follow the same error handling pattern as the other functions here."

The difference is context. The AI that sees your existing code produces something that fits your system. The AI that doesn't see your existing code produces something generic that you'll spend an hour integrating.

The three-step debug loop

When something breaks:

- **Identify** — reproduce the problem reliably. Know exactly what input produces what wrong output.
- **Show** — give the AI the exact error, the exact code, the exact expected behavior:

  > "This function returns [X] when I pass [Y]. I expected [Z]. Here's the code. Why is it returning X and what should I change?"
- **Verify** — don't just accept the fix. Understand it. Ask "why does this fix work?" before applying it. A fix you don't understand is a future bug you won't be able to find.

When to write a comment instead of a prompt

One underused technique: write what you want as a code comment, then ask the AI to implement it.

```
// TODO: validate that the email field is a valid email format // before saving
```

> "Implement the TODO comment in this function."

This approach is useful because it forces you to think precisely about what you want before you prompt — and it leaves a trail in the code of what each piece is supposed to do.

Power user habits for better results

These are the habits that separate people who get consistently great results from code assistants from those who keep hitting walls.

Use Plan Mode first — before writing any code. Most code assistants (Cursor's Composer, Antigravity's Manager View, Claude Code) have a planning or "think" mode that generates a step-by-step implementation plan *before* touching any code. Always use this on anything non-trivial. Ask the tool: "Plan how you would implement this. Don't write any code yet — just describe the approach, which files would change, and what decisions need to be made." Review the plan. Correct misunderstandings. *Then* execute. This single habit eliminates most mid-task derailments.

Use a CLAUDE.md or AGENTS.md file. Cursor, Claude Code, and OpenAI Codex all support a special file at the root of your project (`.cursorrules` , `CLAUDE.md` , `AGENTS.md`) where you document how your project works — its conventions, which libraries to use, what patterns to follow, what to avoid. The AI reads this at the start of every session. A good project file might say: "This is a Next.js app using Tailwind for styling. Always use the existing `useAuth` hook for authentication — never roll your own. Use TypeScript strict mode. Keep components under 150 lines." This context compounds over time — every session starts smarter.

Keep tasks small and atomic. A task like "build the entire user settings page" is too large. Break it down: "Add a form with fields for display name and email" → test → "Add validation to the form" → test → "Connect the form to the save endpoint" → test. Smaller tasks produce more reliable outputs, are easier to review, and fail in more recoverable ways. The temptation to give the AI a big task is strong because the tools *can* handle them — but smaller tasks compound to better codebases.

Use multiple models for different jobs. No single model is best at everything. A common power workflow: use a fast model (Claude Sonnet 4.6, GPT-5.2) for quick edits, explanations, and small generation tasks. Use a powerful reasoning model (Claude Opus 4.6) for architecture decisions, tricky bugs, or anything where careful thinking matters. In Antigravity, you can literally assign different models to different parallel agents. Match the model to the task rather than defaulting to the same one for everything.

After a long session, start fresh with a summary. Code assistant context windows fill up over long sessions. When a session gets long and the AI starts making mistakes it

wasn't making before, that's the signal. Don't keep pushing — start a new session. Open the new session with a summary: "We were building a user authentication system. So far we've completed: [list]. The next step is [task]. Here are the key files: [paste relevant code]." A fresh model with a good summary outperforms a tired model with a full context window.

Run `/clear` or reset when the AI starts going in circles. Every code assistant has some form of context reset. When the AI seems confused — making the same mistake repeatedly, contradicting earlier decisions, or suggesting changes that would break things you already fixed — reset the context rather than trying to argue it out of the confusion. Paste just the relevant code and re-state the problem cleanly. Less conversation, more precision.

Don't accept a change you didn't review. The single biggest risk with code assistants is cargo-cult acceptance — applying changes without understanding them. Develop the habit of reading every diff before applying it. Not necessarily understanding every line, but knowing *what changed* and *why*. Ask the assistant to summarize each change in plain English before you accept it. This keeps you in the driver's seat and prevents small errors from compounding into big ones.

Understanding the code you build

Here's the honest question every vibe coder eventually faces: do I need to understand the code my AI generates?

The answer is nuanced.

You don't need to understand every line. You don't need to know how the routing library works internally, or how the database ORM generates SQL, or how the authentication library handles token validation. These are solved problems. Trust them.

You do need to understand the shape of your system. Where does data come from? How does it flow through the application? What happens when a user submits a form? What calls what? This conceptual understanding — the architecture, not the implementation — is what lets you extend, debug, and make good decisions about your code.

You especially need to understand the code you change. Changing code you don't understand is how vibe-coded projects accumulate debt and become unmaintainable. Every time you make a change, understand what it does before it goes in. Use the code assistant to explain it to you if needed.

The goal isn't fluency in writing code. It's literacy in reading and reasoning about it. That's learnable — and code assistants are remarkably good teachers when you ask them to explain rather than just generate.

When to hand off to a developer

Code assistants extend how far a non-developer can go. They don't eliminate the need for developers entirely. Here's when to bring one in:

Security architecture — anything involving authentication, authorization, data encryption, or payment processing should be reviewed by someone who understands the attack surface. AI generates plausible-looking security code that can have subtle vulnerabilities.

Performance at scale — optimizing for 10,000 concurrent users involves database indexing, caching strategies, and infrastructure decisions that require genuine expertise. Code assistants can help you understand these, but shouldn't be your only guidance.

Integration with complex external systems — connecting to enterprise APIs, legacy systems, or complex data pipelines often involves undocumented edge cases and organizational knowledge that no AI has.

When the codebase exceeds your ability to reason about it — if you can no longer describe what your system does, how its pieces relate, and why something might be failing, that's the signal. Bring in a developer not to take over, but to help you restore that understanding.

The handoff is collaborative, not a surrender. A developer reviewing and extending a vibe-coded foundation is much more productive than a developer building from scratch — especially when you can explain what you built and why.

What to take from this

Code assistants are the bridge between vibe coding and professional development. Not a replacement for either — the natural next step when you need more visibility and control than vibe coding provides, without going fully developer.

Context is everything. Always show existing code before asking for new code. AI that sees your system generates code that fits it. The AI that doesn't generates something generic you'll spend time integrating.

Understand what you change. Extending code you don't understand is how projects become unmaintainable. Use the AI to explain before you apply. You need code literacy more than code fluency — the ability to read it, reason about it, and understand the shape of your system.

Know when to bring in a developer. Security, scale, complex integrations, and codebases you can no longer reason about are the signals. The handoff is collaborative, not a failure.

Want the full framework?

This post covers working with code assistants. The *AI Development Guide* by Jaehee Song goes deeper — into how to set up an effective AI-assisted development workflow, how to use code assistants for specific domains (data pipelines, APIs, frontend, mobile), and how to maintain code quality over time when AI is generating most of it.

🛒 Amazon 📱 Apple Books ▶️ Google Play Books 🌐 All Platforms (Books2Read)

Next in the series: "From Demo to Production" — how to take what you've built and make it reliable, cost-effective, and ready for real users at scale.

Part 10

From Demo to Production

13 min read · Shipping

Part 10 of the "Build with AI" series

> **In short:** "It worked in the demo" is not "it works in production." A demo is optimized for the best case; production must survive unpredictable inputs, real users' data, cost limits, and failures that damage trust. Production-readiness moves through five stages — from a controlled prototype to a system that holds up when reality varies wildly.

There's a graveyard most people never talk about.

It's filled with AI projects that worked beautifully in the demo. The founder showed it to investors. The team applauded. The prototype ran flawlessly on a prepared dataset with three test users. Everyone was excited.

Then it went live. Real users. Real data. Real scale. Real edge cases. And one by one, the cracks appeared.

The AI gave confident wrong answers to paying customers. The costs ballooned because nobody had calculated what 10,000 API calls a day would cost. The system slowed to a crawl under actual load. A user found an input that broke the whole thing. The responses that sounded great in testing sounded off in production — because real users asked different questions than the team anticipated.

The demo worked. The product didn't.

This is the gap most AI builders fall into — and it's not because they're bad at building. It's because demo-thinking and production-thinking are fundamentally different disciplines. This post is about production-thinking.

The demo-production gap

A demo is optimized for showing the best case. A production system has to handle every case — including the ones you didn't anticipate.

Here's what changes when you go from demo to production:

| Demo | Production | |------|------------| | Curated inputs | Unpredictable inputs | | Your data | Users' data | | 3 test users | Thousands of users | | You know what to expect | Users surprise you constantly | | Cost doesn't matter | Cost determines viability | | Failures are invisible | Failures damage trust | | You control the context | Context varies wildly | | Speed is acceptable | Speed is a feature |

None of these transitions are impossible. But each one requires deliberate design. None of them happen automatically because your demo worked.

Stage 1: from prototype to pilot

The first transition isn't from demo to full production — it's from demo to a controlled pilot with real users.

A pilot is small enough that you can watch everything, fix things manually when they break, and learn what production will actually look like before you're committed to it at scale.

What a good pilot looks like:

- 10–50 real users, not test accounts
- Real data, not curated examples
- Real tasks, not scripted scenarios
- A feedback mechanism so users can tell you when something goes wrong
- You or someone on your team watching outputs regularly

What you're learning in the pilot:

- What do real users actually ask or input? (Almost always different from what you expected)
- Where does the AI fail, hallucinate, or produce outputs that feel wrong?
- How fast does it need to be? (Users have much less patience than developers)
- What does it actually cost at real usage volumes?
- What edge cases appear that you never anticipated?

Don't skip the pilot by going straight to a full launch. The pilot is where you learn what production actually requires — at a cost you can still absorb.

Stage 2: reliability engineering

Production systems fail. The question isn't whether — it's when, how, and what happens when they do.

Handling AI failures gracefully

AI outputs are probabilistic. They will sometimes be wrong, incomplete, or inappropriate. Your system needs to handle this without breaking or embarrassing you.

Validation layers: don't pass raw AI output directly to users for anything consequential. Build validation that checks the output makes sense before delivering it. Does it have the expected format? Does it reference things that exist? Is the length reasonable?

Fallback paths: what happens when the AI fails, times out, or returns something invalid? Have a plan. A graceful fallback — "We weren't able to generate a response, here's how to reach us directly" — is infinitely better than a broken experience.

Human-in-the-loop for high stakes: for anything where being wrong has real consequences — a medical question, a legal detail, a financial recommendation — build a human review step. AI drafts. Human approves. The cost of review is much lower than the cost of a consequential mistake.

Latency: speed is a feature

Users are impatient. The average person will abandon a process if it takes more than 3-4 seconds to respond. AI inference — especially for complex reasoning tasks — can be slow.

Strategies:

- **Streaming responses** — show the response as it generates, character by character. Users perceive streamed responses as faster even if the total generation time is the same.
- **Caching** — if the same or similar questions get asked repeatedly, cache the responses. Don't regenerate what you've already generated.
- **Model tiering** — use a faster, cheaper model for simple tasks. Reserve your most capable (and slowest) model for tasks that genuinely need it.
- **Async processing** — for tasks that don't need an immediate response (report generation, batch analysis), process in the background and notify when ready.

Monitoring: you can't fix what you can't see

In production, you need visibility into what's happening at all times.

What to monitor:

- Response latency (how long are requests taking?)
- Error rates (how often are things failing?)
- AI output quality (are the responses actually good? This requires sampling and human review, not just automated metrics)
- Cost per request (are you spending what you expected?)
- Usage patterns (who is using what, when, and how?)

Tools like **Langfuse**, **Helicone**, and **Langsmith** are built specifically for monitoring AI applications — they log every prompt, every response, every latency measurement, and give you dashboards to spot problems before users tell you about them.

Instrument everything before you launch. Retrofitting monitoring onto a live system is painful and means you operated blind during the period you most needed visibility.

Stage 3: cost architecture

This is the stage that kills the most AI projects — not because the product doesn't work, but because the economics don't.

AI API calls cost money. At demo scale, the cost is trivial — a few dollars a month. At production scale, with thousands of users making dozens of requests each, the numbers can become very uncomfortable very quickly.

The cost calculation nobody does in advance

Before you launch, calculate your cost per user per month.

The formula:

```
Cost per request = (input tokens × input price) + (output tokens × output price)
```

Then ask: at your planned pricing, does this leave a viable margin?

Real numbers to work with (approximate, mid-2026):

- Claude Sonnet 4.6: ~$3 per million input tokens, ~$15 per million output tokens

- GPT-5.2: ~$10 per million input tokens, ~$40 per million output tokens
- Claude Haiku 4.5: ~$0.25 per million input tokens, ~$1.25 per million output tokens

A typical chat interaction might use 1,000 input tokens and 500 output tokens. At Sonnet 4.6 prices, that's about $0.003 per interaction — $0.30 for 100 interactions. Manageable.

But if your system includes large context (long documents, long conversation histories), complex reasoning tasks, or many sequential agent steps, costs multiply fast. Always run the numbers before you commit to a model.

Cost control strategies

Prompt compression: shorter prompts cost less. Remove unnecessary context. Summarize long histories instead of passing them in full. Every token saved is money saved.

Model tiering: route simple tasks to cheap models, complex tasks to capable ones. A classification task that costs $0.001 on Haiku doesn't need to run on Opus.

Caching: cache common responses. Cache expensive computations. Cache anything that's asked repeatedly.

Context window management: long context windows are expensive. Don't include everything — include what's relevant. Build retrieval systems that fetch relevant context rather than loading everything.

Batch processing: many APIs offer batch pricing at 50% discount for non-real-time requests. If your use case doesn't need immediate responses, batch everything.

Usage limits: set per-user limits. Not because you're stingy — because a single runaway usage pattern can generate unexpected costs before you've had a chance to notice.

Stage 4: scaling architecture

What works for 100 users often breaks for 10,000. Not because the AI is different — because the infrastructure around it isn't designed for load.

Stateless vs. stateful design

AI interactions are most simply designed as stateless: each request is independent, contains all the context it needs, and doesn't depend on server-side state from

previous requests. Stateless systems scale horizontally — add more servers, handle more requests.

Stateful systems — where the server maintains conversation history, user context, or ongoing agent state — are harder to scale. If you're building stateful AI features, use a proper database for state, not server memory.

Rate limits are your friend

Every AI provider enforces rate limits — maximum requests per minute. At low usage, you never hit them. At production scale, you might.

Design your system to handle rate limit errors gracefully:

- Queue requests when you're near the limit
- Implement exponential backoff (retry after 1s, then 2s, then 4s, then 8s...)
- Use multiple API keys for different workloads if your usage justifies it

The prompt is infrastructure

In production, your prompts are not just instructions — they're code. They need version control, testing, and deployment processes just like your application code.

When you change a prompt:

- Test the change against a representative sample of real inputs
- Compare the new outputs against the old ones
- Deploy gradually — A/B test the change on a portion of traffic before rolling it out fully
- Be able to roll back instantly if something goes wrong

Teams that treat prompts as throwaway text and change them casually in production break things in hard-to-debug ways.

Stage 5: the production mindset

Beyond the technical requirements, there's a mindset shift that separates builders who ship production AI from those who ship demos.

Ship early, learn continuously

The best production systems aren't designed perfectly upfront — they're improved continuously based on real usage. Ship the simplest version that works. Watch what happens. Fix what breaks. Add what's needed.

The worst mistake is waiting until everything is perfect. Production will reveal problems your testing never found — that's not a failure of your testing, it's the nature of complex systems.

Build for the users you have, not the users you imagined

Real users are different from imagined users. They have different vocabularies, different expectations, different use patterns. They misuse things in creative ways. They ask questions you never anticipated.

The fastest way to learn what your users actually need is to watch them use your system — not survey them, not user-test them, but watch the real interactions in your logs. What they do tells you more than what they say they'll do.

Trust degrades fast and rebuilds slowly

In consumer AI products especially, trust is the key asset. Users who have a bad experience — a confidently wrong answer, a broken response, an inappropriate output — don't just report it. They leave. And they tell others.

Build conservatively. It's better to be slightly less capable and reliably good than to be impressively capable and occasionally terrible. Users remember the terrible much longer than the impressive.

The production checklist

Before you move from demo to production, work through this:

Reliability

- [ ] Validation layer on AI outputs
- [ ] Graceful fallback when AI fails
- [ ] Human review for high-stakes outputs
- [ ] Error handling with meaningful user messages

Performance

- [ ] Streaming responses where applicable
- [ ] Response caching strategy

- [ ] Model tiering for different task complexity
- [ ] Latency targets defined and tested

Cost

- [ ] Cost per user per month calculated
- [ ] Margin verified at target scale
- [ ] Model selection optimized for cost/quality
- [ ] Usage limits and alerts configured

Monitoring

- [ ] Latency monitoring active
- [ ] Error rate monitoring active
- [ ] Cost monitoring with alerts active
- [ ] Output quality sampling process defined

Scaling

- [ ] Stateless design verified
- [ ] Rate limit handling implemented
- [ ] Prompts under version control
- [ ] Deployment and rollback process tested

What to take from this

Demo-thinking and production-thinking are fundamentally different disciplines. A demo shows the best case. Production handles every case. Design for every case from the start.

Always pilot before you launch. Ten to fifty real users with real data will teach you more about what production requires than 1,000 hours of testing with curated inputs.

Calculate the cost before you commit to the model. Cost per user per month, at your planned pricing, at realistic usage volume — run these numbers before you ship, not after you're surprised by an invoice.

Instrument everything before launch. Latency, errors, costs, output quality. You can't fix what you can't see.

Treat prompts as infrastructure, not instructions. Version control, testing, gradual deployment, rollback. Prompts that change casually in production are a production incident waiting to happen.

> **Taking a demo to production?** That's exactly where the follow-up guide begins. Beyond Vibe Coding walks through hosting, secrets, legal basics, analytics, and the operations muscle to keep a real service running — then how to validate demand and charge your first dollar.

Want the full framework?

This post covers the production lifecycle. The *AI Development Guide* by Jaehee Song goes deeper — into specific architectural patterns for different types of AI applications, how to build evaluation frameworks for AI output quality, and how to scale from your first hundred users to your first hundred thousand.

🛒 Amazon 📱 Apple Books ▶️ Google Play Books 🌐 All Platforms (Books2Read)

Next in the series: "Risk, Hallucination & Responsible AI" — what every builder needs to know about AI's failure modes, legal exposure, and how to build systems that are trustworthy by design.

Part 11

Risk, Hallucination and Responsible AI

13 min read · Shipping

Part 11 of the "Build with AI" series

> **In short:** A hallucination is when AI states something false, fabricated, or unsupported in the same confident tone it uses for accurate information — it has no internal flag for uncertainty; it's just completing a pattern. This post breaks AI's failure modes into four recognizable hallucination types and gives you a practical safeguard toolkit to catch them before they ship.

There's a moment every AI builder eventually faces.

A user comes to you and says: "Your AI told me [something wrong]. I acted on it. It cost me."

Maybe it was a confidently stated fact that turned out to be false. Maybe it was advice that didn't apply to their situation. Maybe it was an output that felt right in testing but was deeply inappropriate for this specific person in this specific moment.

You built something. It caused harm. And you're responsible for it — even if the AI said it, not you.

This post is about taking that responsibility seriously before you ship, not after. Not because building with AI is dangerous — it isn't, most of the time. But because the builders who think carefully about risk build better products. They design safeguards that users never notice because they never need to. They make judgment calls in advance about where AI belongs in their system and where humans need to stay in the loop.

That's what this post covers.

Understanding hallucination: the core risk

We touched on hallucination in Post 2. Here we go deeper — because if you're building something real, you need to understand exactly what you're dealing with.

Hallucination is when an AI model generates content that is factually incorrect, fabricated, or unsupported — but presents it with the same confident tone it uses for accurate information. The model doesn't know it's wrong. There's no internal flag that says "I'm uncertain here." The output just flows.

Why it happens

Language models predict the most statistically likely next token given everything that came before. When the training data contains enough examples of a pattern, the model completes that pattern confidently. When it doesn't — when asked about something obscure, recent, or outside its training — it completes the pattern with whatever fits grammatically and contextually, which may bear little relationship to reality.

It's not lying. It has no concept of truth vs. falsehood in the way humans do. It's completing a pattern.

The four hallucination types

Factual hallucination — stating false facts with confidence. Dates, statistics, citations, names, events. The most common and easiest to catch — if you check.

Reasoning hallucination — producing a logically flawed chain of reasoning that arrives at a wrong conclusion, presented as sound logic. Harder to catch because it sounds plausible.

Source hallucination — inventing citations, papers, articles, or quotes that don't exist. Particularly dangerous in research, legal, or medical contexts.

Contextual hallucination — generating content that is technically accurate in isolation but wrong for the specific context — the specific user, the specific situation, the specific jurisdiction or domain.

When hallucination is dangerous

Hallucination is not equally dangerous in all applications.

An AI that generates marketing copy occasionally produces a phrase that's slightly off? Easy to catch. Low stakes.

An AI that provides medical information and confidently states the wrong dosage? Potentially life-threatening.

An AI that provides legal guidance and cites a case that doesn't exist? Professionally ruinous for the person who relies on it.

An AI that provides financial advice and states a return that's fabricated? Financially damaging.

The risk of hallucination scales with two factors: **how consequential the output is** and **how likely the user is to verify it independently**. When both are high stakes and low verification — medical, legal, financial, safety-critical — you need the strongest safeguards. When both are low stakes and easy to verify — marketing copy, brainstorming, drafting — the risk is manageable.

The risk spectrum

Not all AI applications carry the same risk. Understand where yours sits.

Low risk

- Creative content generation (marketing, writing, ideation)
- Summarization of content the user provided
- Internal tools with expert users who will review outputs
- Brainstorming and ideation where outputs are starting points
- Entertainment and low-stakes recommendations

Safeguards needed: basic quality checks, user ability to edit, transparent that it's AI-generated.

Medium risk

- Customer-facing information systems
- Research assistance tools
- Automated communications (emails, messages to real people)
- Classification and routing of real business processes
- Tools that influence decisions without directly making them

Safeguards needed: validation layers, human review for edge cases, clear disclaimers, audit logging, user feedback mechanisms.

High risk

- Medical, legal, or financial guidance
- Safety-critical information (emergency procedures, hazard warnings)
- Automated decisions with significant consequences (hiring, lending, medical triage)
- Tools used by vulnerable populations (mental health, crisis support)
- Content that directly reaches customers without human review

Safeguards needed: human in the loop for all consequential outputs, explicit disclaimers, professional oversight requirements, comprehensive logging, regular audits of output quality, clear escalation paths.

Building safeguards: the practical toolkit

1. Grounding

Grounding means connecting AI outputs to verified, authoritative sources — rather than letting the model generate from its training data alone.

RAG (Retrieval-Augmented Generation) is the most common implementation. Instead of asking the model to recall facts, you retrieve relevant documents from a trusted source first, then ask the model to answer based only on those documents.

The prompt pattern: > "Answer the following question using ONLY the information in the provided documents. If the answer is not contained in the documents, say 'I don't have enough information to answer this.' Do not use your general knowledge. > > Documents: [retrieved content] > Question: [user question]"

This dramatically reduces factual hallucination — the model can't fabricate what it can't find in the source material. It's not foolproof (the model can still misinterpret or misquote), but it's far more reliable than open-ended generation.

2. Confidence signaling

Build systems that communicate uncertainty honestly — so users know when to be more careful about verifying.

Explicit uncertainty: prompt the model to express uncertainty when it has it: > "If you are uncertain about any part of your answer, say so explicitly. Use phrases like 'I'm not certain, but...' or 'You should verify this, but...' rather than stating uncertain things confidently."

Confidence scoring: some applications add a secondary prompt that evaluates the confidence of the primary output: > "On a scale of 1-5, how confident are you in the accuracy of the response you just gave, and why? 1 = very uncertain, 5 = highly confident."

This isn't foolproof — models are poorly calibrated on their own uncertainty — but it catches the most obvious cases where the model is clearly guessing.

3. Scope limiting

Define clear boundaries for what your AI will and won't do — and enforce them consistently.

In-scope/out-of-scope prompting: > "You are a customer support assistant for [Company]. You can help with: account questions, product features, billing, and technical support for our platform. If a user asks about anything outside these topics — including general advice, other companies' products, medical or legal questions — politely redirect them and explain that you're only able to help with [Company]-related questions."

Topic detection: for high-risk topics, add an explicit detection layer: > "Before responding, determine if this question involves medical advice, legal advice, financial advice, or emergency situations. If yes, respond only with a referral to an appropriate professional and do not attempt to answer the question."

4. Output validation

Don't pass raw AI outputs directly to users for consequential things. Build a validation step.

Format validation: did the output have the structure you expected? If your AI should always return a JSON object with specific fields, check that before using it.

Content validation: does the output contain expected elements? Does it avoid prohibited content? Run a secondary check: > "Review the following response. Does it: (1) stay within the topic scope, (2) avoid making specific claims about dosages, legal outcomes, or financial returns, (3) include appropriate caveats for uncertain information? If any of these fail, return FAIL and explain why."

Consistency checking: for high-stakes outputs, generate multiple responses and check for consistency. Significant disagreement between generations signals high uncertainty.

5. Audit logging

Log everything, always. For any production AI system:

- Every input (what the user sent)
- Every output (what the AI returned)
- Timestamps and user identifiers
- Model and prompt version used
- Any flags raised by validation layers

This isn't just good practice — in regulated industries it may be legally required. Practically, it's the only way to investigate problems after they occur, identify systematic failure patterns, and demonstrate due diligence if something goes wrong.

The legal and ethical picture

The regulatory environment around AI is evolving fast. What builders need to know in 2026:

Liability

In most jurisdictions, the legal framework for AI liability is still developing. But the practical reality is already clear: **if your AI product causes harm, you are accountable** — even if the harm was technically caused by the AI's output, not a decision you made directly.

Courts and regulators are increasingly treating AI outputs in consequential domains as equivalent to professional advice or product recommendations — subject to the same standards of care as any professional service.

The practical implication: apply the same level of caution to your AI's outputs that you'd apply to your own professional advice. If you wouldn't say it yourself without qualification, your AI shouldn't say it either.

Transparency

Users have a right to know they're interacting with AI. This is increasingly being codified into law across multiple jurisdictions.

Practical requirements:

- Clearly label AI-generated content as AI-generated
- Don't design AI personas to deceive users into thinking they're human
- Disclose when AI is being used in decisions that affect users (hiring, lending, content moderation)

Data privacy

When users interact with your AI system, they often share personal information — sometimes sensitive personal information. This creates obligations:

- What data is retained? For how long?
- Is user data being used to train or fine-tune models?
- Are you compliant with GDPR, CCPA, or other applicable regulations?
- Do users have the right to request deletion of their data?

Read the terms of service of every AI API provider you use. Some providers allow your data to be used for model training by default — you need to opt out if that's not acceptable.

Bias and fairness

AI models trained on historical data can reproduce and amplify historical biases. In consequential applications — hiring tools, loan approval, content moderation — this creates both ethical problems and legal risk.

Practical steps:

- Test your system across diverse user groups before deploying
- Monitor outputs for systematic differences across demographic groups
- Build feedback mechanisms so users can flag problematic outputs
- Don't deploy AI in high-stakes decision-making without understanding its error patterns

---

The responsible builder's mindset

Beyond the technical and legal requirements, there's a deeper question every AI builder should sit with:

What could go wrong with what I'm building — and am I comfortable with that?

Not "is there any possible way this could cause harm" — of course there is, for almost anything. But: have I thought carefully about the realistic failure modes? Have I designed for them? Am I confident that the benefit I'm creating outweighs the risk I'm creating?

This is the same question a doctor asks before prescribing medication, a lawyer before giving advice, an engineer before signing off on a design. Professional

responsibility.

The AI field is young enough that this culture of responsibility is still forming. The builders who shape it well — who take the failure modes seriously, who design safeguards, who communicate honestly about limitations — will build things that last and that users can actually trust.

The ones who ship fast without thinking carefully about failure modes will create the cautionary tales that justify regulation.

Build like the first group.

The red flags checklist

Before you ship, ask yourself:

About your outputs:

- [ ] Have I tested with inputs designed to break or mislead the system?
- [ ] Do I know what the system says when it's uncertain?
- [ ] Have I tested with edge-case users (non-native speakers, users with accessibility needs, users in crisis)?
- [ ] Are outputs validated before reaching users?
- [ ] Is there a human review step for high-stakes outputs?

About trust and transparency:

- [ ] Do users know they're interacting with AI?
- [ ] Are limitations and disclaimers clearly communicated?
- [ ] Is there a clear feedback path when something goes wrong?
- [ ] Are consequential AI decisions explainable to users?

About data:

- [ ] Do I know what data is retained and for how long?
- [ ] Have I read the data policies of every API provider I use?
- [ ] Is user data handled in compliance with applicable regulations?
- [ ] Do users have appropriate control over their data?

About failure:

- [ ] Is everything logged?

- [ ] Do I have monitoring that will alert me when something goes wrong?
- [ ] Is there a process for investigating and addressing problems?
- [ ] Can I roll back or disable the system quickly if needed?

What to take from this

Hallucination is structural, not a bug to be fixed. Models generate the most likely next token — they don't verify truth. Design your system assuming outputs will sometimes be wrong, and build accordingly.

Risk scales with consequences and verification likelihood. Medical, legal, financial — highest risk. Creative, brainstorming, internal tools — lowest risk. Know where yours sits and design safeguards proportional to the stakes.

Ground, scope, validate. RAG for factual grounding. Clear in/out-of-scope definitions. Output validation before user delivery. These three together eliminate most of the common failure modes.

Log everything, always. You can't investigate what you didn't record. Logging is the foundation of accountability, quality improvement, and legal defensibility.

The responsible builder question: have I thought carefully about what could go wrong? Not "is there any risk" — yes, always. But: have I designed for the realistic failure modes? Is the benefit worth the risk? Am I comfortable with what I'm shipping?

Want the full framework?

This post covers the risk and responsibility layer. The *AI Development Guide* by Jaehee Song goes deeper — into specific safeguard architectures for different application types, how to build evaluation frameworks for AI quality, and how to navigate the evolving regulatory landscape across jurisdictions.

🛒 Amazon 📱 Apple Books ▶️ Google Play Books 🌐 All Platforms (Books2Read)

Final post in the series: "The Next Wave — AI Agents, Physical AI, and What Comes After" — where the technology is heading and what builders should be preparing for now.

Part 12

The Next Wave: AI Agents, Physical AI, and What Comes After

13 min read · Future

Part 12 of the "Build with AI" series — The Final Post

> **In short:** The next wave of AI is already forming: autonomous agent networks that coordinate multiple specialized agents, physical AI and humanoid robots that act in the real world, and early self-improving systems. Today's baseline — operational agents, narrow AI "employees," and a nearly closed software-development loop — is the launch point, not the destination.

This series started with frustration — the gap between what AI can supposedly do and what most people are actually getting from it. We've spent eleven posts closing that gap: understanding how AI thinks, building with data and prompts, automating with agents, shipping to production, and building responsibly.

Now we look forward.

Because the gap isn't standing still. The tools are evolving faster than almost any technology in history. And the next wave isn't a refinement of what we've been discussing — it's a different category of capability entirely.

This post is about what's coming. Not to predict the future with false precision, but to give you a map of the territory so you can position yourself intelligently. The builders who understand what's happening now will have a real advantage as the next wave arrives.

Where we are right now

Before looking forward, it's worth anchoring on what's actually true today — not hype, not speculation.

Agentic AI is operational. The multi-step, tool-using, decision-making agents we discussed in Post 7 are not experimental. They're being deployed at scale in enterprise workflows, customer service, software development, sales, and research. Anthropic's Claude Code, OpenAI's Operator, and Google's Gemini are all running autonomous multi-step tasks routinely.

AI employees are real, in narrow domains. Companies like Artisan, 11x, and Relevance AI have deployed AI SDRs — sales development representatives — that prospect, research, write outreach, follow up, and book meetings autonomously. They're not perfect. They break on edge cases. But they're running in production for real companies generating real revenue.

The software development loop is closing. As of early 2026, Claude Code is widely considered the best AI coding assistant, handling full codebases, multi-file edits, and agentic coding tasks that would have required senior engineers a year ago. The gap between "describing software" and "having software" has collapsed to hours in many domains.

This is the baseline. Everything that follows is what's happening next.

Wave 1 already breaking: autonomous agent networks

The first shift already underway is the move from single agents to networks of agents — systems where multiple AI agents work together, each specialized, coordinating toward shared goals.

During the next decade, the intersection of agentic AI systems with physical AI robotic systems will result in robots whose "brains" are agentic AIs — able to adapt to new environments, plan multi-step tasks, recover from failure, and operate under uncertainty.

But in pure software, it's already here. The pattern: an **orchestrator agent** receives a goal, breaks it into subtasks, assigns each to a **specialist agent**, coordinates the results, and delivers the output. The specialist agents might handle research, writing, fact-checking, formatting, and quality review — each optimized for its task, running in parallel.

What this makes possible in practice:

Research at unprecedented speed. A swarm that simultaneously searches dozens of sources, extracts structured data from each, cross-references for consistency, and

synthesizes into a report — completing in minutes what would take a human researcher days.

Software teams in miniature. Agent networks that handle the full software development loop: requirements analysis, architecture planning, implementation, testing, debugging, documentation, and deployment — with a human reviewing and approving at key checkpoints.

Business process automation at depth. Not just routing emails or filling forms, but handling the full lifecycle of a business process: intake, analysis, decision, action, follow-up, exception handling — with humans intervening only for genuinely novel situations.

If you're designing agent systems today, design them to be composable. Build agents that can call other agents. Build orchestration layers that can coordinate them. The systems that will scale are the ones built as networks, not monoliths.

Wave 2 building: physical AI

The most dramatic development of 2026 — and the one with the longest reach — is the convergence of AI intelligence with physical hardware. Physical AI.

At CES in Las Vegas, NVIDIA CEO Jensen Huang said the "ChatGPT moment for physical AI is here," marking an inflection point in the robotics space.

What does that actually mean?

For decades, robots have been scripted machines. They follow pre-programmed routines — precise, reliable within their defined parameters, useless outside them. A factory robot that welds the same joint 10,000 times a day is extraordinarily capable at that exact task and incapable of anything else.

The shift: AI-powered robots can now perceive their environment, understand natural language instructions, reason about what to do next, and adapt to situations they've never encountered before. They're not following a script — they're thinking.

The enabling technology is **VLA models** — Vision-Language-Action models that integrate visual perception (seeing the environment), language understanding (processing verbal commands), and action execution (moving in the physical world). VLA models enable robots to interpret visual inputs and language commands, transforming them into actionable tasks — replacing outdated scripted routines.

Who is building physical AI right now

NVIDIA's Isaac GR00T N1.6 is an open reasoning vision language action model, purpose-built for humanoid robots, that unlocks full body control and uses NVIDIA Cosmos Reason for better reasoning and contextual understanding. ABB Robotics, AGIBOT, Agility, Figure, KUKA, Universal Robots, and YASKAWA are among the leaders building on NVIDIA technology to deploy physical AI at scale.

Tesla's Optimus Gen 2 is designed to handle repetitive tasks, assist in manufacturing, and perform home automation functions — learning from real-world data. Boston Dynamics unveiled its all-new Electric Atlas at CES 2026, a high-performance, enterprise-grade humanoid designed for industrial tasks from material handling to order fulfillment. 1X has opened preorders for its NEO home humanoid, with first customer deliveries planned for 2026.

Chinese humanoid robot makers showed off their wares at Smart Factory & Automation World in Seoul in March 2026 — the entire COEX venue, 2,300 booths. AGIBOT has announced the rollout of its 10,000th humanoid robot, becoming one of the first companies in the industry to reach this milestone at scale.

Where physical AI is already working

The robotics field in 2026 has moved from experimental novelty to active pilot programs in warehouses and factories.

Amazon's warehouse fleet crossed 1 million robots in 2025, with its new DeepFleet AI model expected to boost travel efficiency by 10% across the network. Surgical robotics have reached 60% adoption in large hospitals, with robotic-assisted procedures now accounting for 55% of complex surgeries in developed nations.

A Deloitte survey of more than 3,200 global business leaders found about 58% are currently using physical AI to some extent in operations, growing to 80% who plan to within the next two years.

Where physical AI still struggles

Honesty matters here. Most humanoid platforms still struggle with an "operational ceiling" of three to four hours of battery life, and dexterity for delicate tasks — like threading a needle or handling fragile items — still lags behind human capability.

Despite achieving 95% accuracy in controlled environments, performance can drop to 60% in actual conditions due to environmental factors.

Physical AI is real and accelerating, but the gap between lab performance and field performance remains significant. The honest picture: physical AI is transforming warehouses, factories, and surgical suites now. It's coming to service industries and homes in the next five years. But the vision of a fully capable general-purpose robot doing everything a human can do remains a decade or more away.

What physical AI means for Korea

This is particularly relevant for readers building in the Korean technology ecosystem.

Korea is among the world's leaders in robotics manufacturing, semiconductor technology, and smart factory deployment. South Korea-based startup RLWRLD is developing foundational AI models that allow traditional manual-intensive processes to be performed autonomously by robots through automated learning and mimicking human expertise.

The Korean government's smart city and smart factory initiatives — many of which are part of the very ecosystem this series was written alongside — are a direct entry point into the physical AI wave. The K-startup ecosystem has an unusual opportunity: deep hardware manufacturing expertise, government support for smart infrastructure, and proximity to the Asian markets where physical AI deployment will scale fastest.

Wave 3 on the horizon: AI that improves itself

The third wave is the most speculative — but also the most consequential if it arrives as some believe it will.

Current AI systems are trained, deployed, and then static. Claude Opus 4.6 doesn't get smarter from your conversations with it. GPT-5.2 learns nothing from its interactions in deployment. The model you use today is the same model you'll use in six months — unless Anthropic or OpenAI releases an update.

The next frontier: AI systems that improve themselves through interaction — that learn from each deployment, each failure, each new piece of information — and get measurably better over time without retraining from scratch.

This isn't happening at scale yet. But the research directions are clear:

Continual learning — systems that update their weights from new data without catastrophic forgetting of what they already knew.

Self-play and self-improvement — systems that generate their own training signal by competing against previous versions of themselves (the approach that made AlphaGo superhuman at chess applies to reasoning too).

Recursive improvement — systems that help improve their own training process, generating better data, better evaluation criteria, better fine-tuning approaches.

If this wave arrives, it changes the fundamental dynamic of the field. Right now, improvements come in discrete jumps — model releases every six to twelve months,

each significantly better than the last. Self-improving systems would make progress continuous rather than discrete — always getting better, always adapting.

The implications for builders: the systems you build on top of would improve without you doing anything. But the systems you compete with would improve without anyone doing anything either. The moat would have to come from your data, your domain expertise, your relationships — not the underlying model capability, which everyone would have access to at roughly the same rate.

What this means for builders today

Three concrete implications from everything above:

1. Your domain knowledge is the moat

As the tools get better at the generic parts — writing code, generating content, reasoning about problems — the value shifts to what only you know. Your industry expertise. Your customer relationships. Your understanding of a specific domain's nuances. Your proprietary data.

A generic AI system can help anyone write a marketing email. A system trained on your 20 years of enterprise data engineering experience and 500 students of vibe coding curriculum — that's not generic. The builders who win will be the ones who combine powerful generic AI with deep domain specificity.

2. Build for adaptability, not just for now

The tools will change. A workflow you build today on Claude Opus 4.6 will be available to run on whatever model is 10x more capable in two years. Design your systems so that "swap the model" is easy. Don't hard-code prompts with model-specific quirks. Don't build workflows that only work with one provider's APIs.

The abstraction layer — the logic, the data, the workflow design — is yours. The model underneath is a commodity that gets better over time. Treat it that way.

3. The best time to build your AI fluency is now

Here is the honest truth about the next wave: it will be significantly more powerful than what we have today. And it will make people who already understand the fundamentals — problem definition, data quality, prompting, agents, production deployment, responsible building — dramatically more effective than people starting from zero.

The gap between "understands AI fundamentals" and "doesn't" will widen as the tools get more powerful, not close. More powerful tools in the hands of someone who doesn't understand how to direct them just produces faster mediocrity. More powerful tools in the hands of someone who does understand them compounds their advantage.

You've read this series. You're not starting from zero anymore.

A word on what this doesn't mean

The next wave will generate a lot of fear alongside the excitement. Jobs displaced. Power concentrated. Systems beyond human oversight.

Some of those concerns are legitimate and deserve serious attention — this is not the place to dismiss them. The post on responsible AI (Post 11) covers the builder's obligations in this environment. The regulatory conversation, the alignment research, the labor economics — these are real discussions happening in real places, and they matter.

But for a builder reading this: the technology itself is not the determinant of whether the wave is good or bad. How it's deployed is. Who it serves is. What safeguards are built in is. Those are decisions made by builders — by you.

The builders who think carefully about these questions, who build with the responsibility framework from Post 11 alongside the technical capability from the rest of this series, are the ones who create things worth creating.

Build that way.

The full picture

Let's close by naming where we started and where we've ended up.

We started with frustration — AI not delivering on its promise, the demo gap, the comparison spiral, feeling left behind.

We've covered:

- The mental model shift (Post 1)
- What AI actually can and can't do (Post 2)
- How the stack fits together (Post 3)

- Why data is the real bottleneck (Post 4)
- Why problem definition is the real skill (Post 5)
- How to prompt with precision (Post 6)
- How to build and use agents (Post 7)
- How to vibe code a real product (Post 8)
- How to go from demo to production (Post 10)
- How to build responsibly (Post 11)
- And now: where the next wave is taking us (Post 12)

The thread running through all of it: **this is learnable.** The tools keep improving. The fundamentals stay the same. The people who get consistently good results from AI — now and in the future — are the ones who think clearly about problems, understand the tools they're using, and build with care.

That's you. Go build.

> **Where to go next:** If you're ready to turn what you've learned into a real product, the follow-up track — Beyond Vibe Coding: From Demo to Real Service — is the practical next step, from launch to first revenue to deciding what your project becomes.

The complete series

If you found this series valuable, the *AI Development Guide* by Jaehee Song is the book it draws from — going deeper on every topic covered here, with practical examples, architectures, and frameworks you won't find scattered across YouTube tutorials and blog posts.

This is the only book that covers the full arc: from AI fundamentals to production deployment, from solo builders to enterprise workflows, from today's tools to where the field is heading.

🛒 Amazon 📱 Apple Books ▶️ Google Play Books 🌐 All Platforms (Books2Read)

Thank you for reading the "Build with AI" series. If these posts have been useful, share them with someone who's frustrated with AI and doesn't know why yet.